OpenURMA: A Clean-Room Open Implementation of the Unified Bus Protocol

Bojie Li

Modern datacenter RDMA is bottlenecked at the network interface, not at the wire. At the scale of AI-training collectives and HPC all-to-alls, a NIC running RoCE or InfiniBand must hold per-connection state for every (application, remote-endpoint) pair — hundreds of megabytes of context at 1024-application fanout — and pays a four-traversal PCIe round trip on the smallest 64-byte operation, inflating end-to-end latency by an order of magnitude beyond the wire delay. Both costs are structural consequences of the Queue-Pair-over-PCIe abstraction that commodity RDMA inherits from InfiniBand.

Huawei's *Unified Bus* (UB) is a public 2025 specification that removes both costs by changing the abstraction. Its transport protocol decouples per-application endpoint state from per-host reliable-transport state, so connection context grows additively rather than multiplicatively; it exposes ordering as an opt-in surface so applications pay gating only when they need it; and it reaches remote memory through a CPU's native load/store instructions to a controller on the on-chip bus rather than through verbs to a PCIe-attached device. The protocol ships in Huawei's Ascend 950 silicon, which is closed.

OpenURMA is the first clean-room open implementation of the Unified Bus protocol's transport and transaction layers, written from the public specification. We realise it at three modeling tiers — synthesisable RTL on Alveo U50, a cycle-level two-node SystemC simulator that accounts for both endpoints of the wire, and a gem5 full-system scaffold in which two ARM CPUs run real user binaries against the SystemC NIC — with a matched OpenRoCE (RoCEv2 RC) baseline at every tier. The contribution is not the architecture, which is Huawei's; it is the implementation, the multi-tier evaluation harness, and the controlled comparison that closed silicon does not admit.

The artifact lets the community measure the protocol's costs cell-by-cell, locate its operating points against RoCE on identical infrastructure, and prototype follow-on transports on a common substrate. We use it to show that UB's three architectural choices are mutually load-bearing: removing any one and keeping the others on a PCIe-attached NIC reproduces RoCE's costs. On a 64-byte READ, the load/store path delivers ≈500 ns end-to-end CPU-to-remote-DRAM-to-CPU latency — **4.37**× below the matched RoCEv2 baseline (2186 ns) — at roughly 14% of a U50's LUT budget.

1. Introduction

A modern datacenter network is two interconnects fused into one. From the application's view, RDMA looks like a load or store to remote memory — bus-like, low-latency, address-and-go. From the NIC's view, every operation is a queued work request shuttled across PCIe to a peripheral device — network-like, with all of the network's per-message machinery and the peripheral's PCIe crossing. For two decades the fusion was tractable: connection counts stayed small, operation sizes stayed large, and the gap between the bus-like view and the network-like implementation could be papered over with cleverer software above the device. AI-training workloads broke both assumptions. A single job now exchanges 64-byte gradient updates across thousands of GPUs; the bus-like view's promises collide with the network-like implementation's costs, and the collision is structural.

The two costs of the collision are both consequences of one design stance: *the NIC is a peripheral*. Peripherals communicate with the CPU through a paired-queue programming model, so the NIC must hold per-pair connection state; peripherals sit behind PCIe, so every operation crosses the PCIe boundary. Under RoCEv2 RC, each (local application, remote endpoint) pair is a Queue Pair (QP) carrying ∼512 B of NIC state; at 1024 of each, the NIC's working set is ∼537 MB, past any commodity NIC's on-chip SRAM and into the regime where every operation pays a PCIe round trip to refetch its QP [20, 15, 17, 42]. The same operation pays a second cost even when state stays on chip: a 64-byte READ spends roughly 1.5 μs of its ∼2 μs round trip on four PCIe traversals (doorbell MMIO, work-queue DMA fetch, completion DMA write, CPU poll-miss across PCIe coherence), while the wire delivers in ∼200 ns. Software cannot move the peripheral; hardware inside the peripheral cannot move PCIe. What removes both costs is moving the peripheral.

Huawei's *Unified Bus* (UB) [12] is a 2025 specification that moves the peripheral. The NIC is no longer behind PCIe but on the on-chip bus, addressed by the CPU through ordinary load and store instructions; connection state is no longer per-application-pair but per-host-pair; ordering is no longer always-on but opt-in. Huawei's Ascend 950 NPU [13] ships UB at commodity scale, but the silicon is closed and the spec is unaccompanied by a public implementation that researchers can measure, instrument, or prototype against.

The three architectural moves UB makes are not independent choices; they form a chain in which each move makes the next possible (Figure 1).

1. **Split the transport layer.** The Queue Pair fused application identity with transport reliability into one object, so state had to scale with the product of the two counts. UB separates the two: per-application end-

point state lives on a *Jetty*; per-remote-host transport state lives on a *TP Channel*. Per-NIC state grows additively, as $O(N+M)$ instead of $O(N \cdot M)$. Bounding state is what makes (2) possible.

2. **Move the controller onto the on-chip bus.** A NIC whose working set fits in on-chip SRAM can live next to the CPU on the on-chip bus rather than behind PCIe; a NIC whose working set spills to host DRAM cannot, because each spill becomes a host-memory access. Putting the controller on-bus is what makes (3) possible.
3. **Admit a load/store data path.** Once the controller is on the on-chip bus, a CPU's load or store instruction can reach it directly. The four PCIe traversals collapse into a single on-chip-bus crossing, and the work-queue and completion-queue machinery elides for small synchronous operations.

A fourth move — opt-in ordering — rides on the same per-application counters the first move provisions, so it costs zero extra pipeline cycles on operations that do not request gating, while letting workloads that need a barrier pay only for that barrier.

OpenURMA is a clean-room open-source implementation of the Unified Bus protocol's transport and transaction layers, written from the public specification. The same artifact is observable at three modeling tiers, each chosen because it is load-bearing for a different sub-claim:

- **Synthesisable RTL on Alveo U50.** For the state and area claims: per-element LUT and BRAM, post-route timing closure, the on-chip working-set size that lets the controller live on-bus, and the gating logic's actual cost in pipeline cycles.
- **A cycle-level two-node SystemC simulator.** For per-op latency and sustained throughput: both endpoints of the wire are modeled concurrently, including the work-queue, doorbell, DMA on both sides, the wire link, DRAM, the cache hierarchy, and the completion path. The simulator runs 388 sweep configurations covering all six verb categories.
- **A gem5 full-system scaffold.** For the CPU-to-remote-DRAM end-to-end claim that requires a CPU in the loop: two ARM CPUs run real user binaries against the SystemC NIC linked into gem5 as a memory-mapped device. The PCIe round trip only exists when a CPU exists; this tier is where it actually appears.

A parallel *OpenRoCE* stack implementing RoCEv2 RC on the same toolchain, same target part, and same test harness anchors an apples-to-apples baseline at every tier. The point of the controlled comparison is to ensure both stacks' numbers come from the same modeling assumptions rather than from published vendor data, which proprietary silicon by construction cannot offer.

The contribution is the implementation and the harness, not the architecture. As a concrete preview: on a 64-byte READ in the two-node simulator, the load/store path delivers ≈500 ns end-to-end at **4.37**× below RoCEv2's 2186 ns.

Contributions.

- **The first clean-room open implementation of the Unified Bus protocol stack.** Synthesisable RTL covering the transport and transaction layers, closing 322 MHz post-route on commodity FPGA at roughly 14% of an Alveo U50's LUT budget.
- **An apples-to-apples OpenRoCE baseline.** A RoCEv2 RC implementation on the same toolchain, same target part, and same test harness, so both stacks' numbers are produced under identical modeling assumptions.
- **Multi-tier evaluation infrastructure.** A cycle-level two-node SystemC simulator that accounts for both endpoints of the wire, plus a gem5 full-system scaffold that places a CPU in the loop — jointly necessary to measure the end-to-end CPU-to-remote-DRAM path the protocol claims to shorten.
- **Ten cross-cutting validation experiments.** Including a model-validation step that reproduces published ConnectX-7 RDMA WRITE latency within ±5%, congestion-control dynamics against DCQCN, and a YCSB-A application port; together these convert UB's analytical claims into measured curves on the open substrate.

OpenURMA is released at `https://github.com/bojieli/OpenURMA`.

2. Background

The introduction stated the thesis at altitude. This section grounds the thesis in concrete RDMA mechanics: what makes the NIC a peripheral, why the peripheral stance imposes the two costs by construction, why three decades of fixes around it leave the abstraction in place, and what Unified Bus replaces it with.

2.1 The peripheral-NIC abstraction

Every commodity RDMA NIC — InfiniBand HCAs, Mellanox/NVIDIA ConnectX, Broadcom Thor, Intel IPU — is a PCIe peripheral. The CPU sees it through a memory-mapped BAR; the NIC sees the CPU through DMA. The pair communicates by a Queue-Pair protocol: the CPU posts work-queue entries to host memory, signals via MMIO doorbells, and the NIC consumes them by DMA and writes completions back the same way. This programming model is itself a small network protocol spoken across PCIe, and the model fixes two properties of the system before the wire is even reached.

The QP fuses what should be two layers. A Queue Pair is named by a (local-application, remote-endpoint) pair and carries both transaction-layer state (work queues, permissions, the identity of *who is calling*) and transport-layer state (sequence numbers, retransmit window, RTO timer, congestion state — the bookkeeping of *how packets are delivered*). Textbook protocol design separates these layers; the QP collapses them into one object whose unit of accounting is the application pair, not

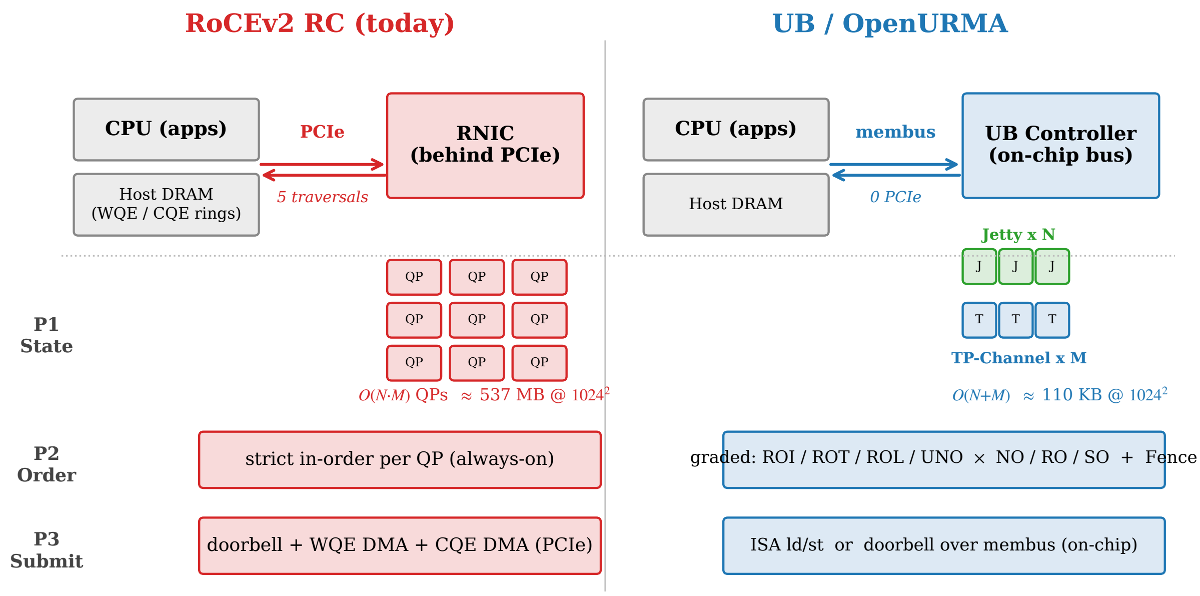


Figure 1: The three architectural moves and their dependencies. RoCEv2 RC (top) puts the NIC behind PCIe, holds one Queue Pair per (application, remote-endpoint) pair, and enforces strict order on every operation. UB (bottom) splits state along the transaction/transport line so it grows as $O(N{+}M)$; places the controller on the on-chip bus, where the CPU can reach it with loads and stores; and exposes ordering as four opt-in axes that reuse counters the layer split already provisions. The arrows mark the chain: bounded state is what lets the controller live on-bus; the on-bus controller is what lets the load/store path exist; the layer split is what makes ordering cheap.

the application count or the peer count. Total NIC state therefore grows as $O(N{\cdot}M)$. At $(N, M){=}(1024, 1024)$, $\sim$1 M QPs at $\sim$512 B each is $\sim$537 MB, beyond any commodity NIC's on-chip SRAM. When the working set spills to host DRAM, every operation pays a PCIe round trip to refetch its QP before it can even consult its sequence number [20, 42].

The PCIe attachment imposes four traversals per operation. Because the NIC is a peripheral, every operation crosses the PCIe boundary four times. On a 64-byte READ: the CPU writes a work-queue entry to host DRAM and then a doorbell to the NIC's MMIO BAR (traversal 1); the NIC DMA-reads the work-queue entry (traversal 2); the NIC emits the wire request, receives the response, and DMA-writes a completion entry to host DRAM (traversal 3); the CPU polls the completion queue and incurs a PCIe-coherence miss to fetch the freshly written line (traversal 4). Of the $\sim$2 μs round-trip time, $\sim$1.5 μs is consumed by these four traversals; the wire itself delivers in $\sim$200 ns. The traversals are not an implementation accident; they are what the model requires when the two communicating parties live in disjoint address spaces.

The two costs are joint consequences of one stance. Both costs follow from treating the NIC as a peripheral. Fixing one in isolation leaves the other in place: connection sharing (XRC, SRQ) reduces state but keeps the peripheral attachment and its four traversals; doorbell coalescing or inline-WQE shortcuts reduce one of the four traversals but keep the QP fusion and its quadratic state. The abstraction itself is the limit on the optimisation budget.

2.2 Why prior fixes are patches

The field has spent three decades attacking the symptoms of the peripheral-NIC abstraction without changing the abstraction itself. The work falls in three buckets. *Software above the device* (FaSST [15], 1RMA [37], Pony Express, the broader SmartNIC-pacing literature) works entirely above the verb interface; it can change the application's view but cannot move the peripheral or rewrite its programming model. *Hardware inside the device* (SRNIC [42], StaR [41], the IRN-derived loss-recovery line [32, 30, 10, 28], MP-RDMA [31], ConWeave [39]) rebuilds the NIC's internals to mitigate one symptom at a time, but the NIC remains a PCIe peripheral with a Queue-Pair programming model and the per-pair state and PCIe traversals are preserved by construction. *Programmable substrates* (Tonic [1], NanoTransport [14], StRoM [36]) provide a fabric on which new transports can be expressed, but they are substrates rather than new transports. None of the three buckets removes both costs.

2.3 Unified Bus: the spec, the silicon, the gap

UB is the first RDMA-class specification since RoCEv2 (2014) to replace the abstraction rather than patch it. The 2025 specification covers physical, link, transport, transaction, and verb layers; this paper engages only the transport and transaction layers, which is where the structural choices live. The protocol ships in Huawei's Ascend 950 NPU [13], which integrates both the work-queue-driven and the load/store-driven data paths as on-die blocks alongside the AI accelerators. The user-space verb library and kernel driver are open-source; the silicon implementation of the protocol itself is closed. OpenURMA fills that gap with a clean-room open implementation built from the public specification.

2.4 Making the NIC a peer of the CPU

The intuition driving every protocol-level choice UB makes is to stop treating the NIC as a peripheral the CPU programs and start treating it as a peer the CPU shares an on-chip bus with. Figure 2 states the move at system altitude. The rest of this subsection walks through the specification-level mechanisms in the order in which each enables the next: the layer split that bounds state, the on-bus controller that becomes feasible once state is bounded, the load/store data path that becomes admissible once the controller is on-bus, and the opt-in ordering surface that the layer split makes cheap.

Move 1: split transaction layer from transport layer. The QP's fusion of transaction-layer and transport-layer responsibilities into one object was a textbook layering violation that paid off only while the quadratic state cost stayed small. UB undoes the fusion explicitly: the transaction-layer responsibilities (application identity, work queues, completion delivery, permissions) live on a per-application object called a *Jetty*; the transport-layer responsibilities (sequence numbers, retransmit window, in-flight bitmap, RTO timer, congestion-control state) live on a per-remote-host object called a *TP Channel* (Figure 3). A Jetty carries $\sim$80 B of state that scales as $O(N)$ in the local application count; a TP Channel carries $\sim$30 B that scales as $O(M)$ in the remote-host count. Any Jetty can address any remote endpoint through the per-host TP Channel pool, with the destination Jetty carried in the packet header rather than baked into a per-pair binding. Total state grows as $O(N+M)$ by construction: at $(N, M)=(1024, 1024)$, roughly 110 KB instead of 537 MB. The quadratic count never appears because the abstraction never asks for it.

The trade-off the split pays is one extra layer of indirection in the NIC pipeline (an outbound packet must look up which TP Channel to ride on by destination host) and a protocol surface that exposes Jetty and TP Channel as separate first-class objects with separate lifecycles. The QP's simplicity — one object per peer, one lifecycle to manage — is the cost of admission.

Move 2: place the controller on the on-chip bus. With per-NIC connection state bounded at $O(N+M)$, the entire working set fits in on-chip SRAM at the scales that previously forced commodity NICs to host DRAM. A NIC whose connection context lives on-chip can in turn live next to the CPU on the on-chip bus rather than behind PCIe: the CPU sees the controller through ordinary memory-mapped regions, and the controller sees the CPU's loads and stores at on-chip-bus latency. The relocation is the architectural payoff of the layer split: it trades the PCIe boundary for a single on-chip-bus crossing and opens the door to the data-path choice below. The trade-off the relocation pays is that the controller must sit on the same on-chip-bus segment as the CPU it serves — it cannot ride across PCIe distance without giving back the latency advantage.

Move 3: admit a load/store data path. Once the controller is on the on-chip bus, UB can define an alternative data path in which a CPU's bare load or store instruction reaches an address aperture owned by the controller (Figure 4). The controller translates each instruction into a single-shot network transaction with no work-queue entry, no doorbell, no DMA, and no separate completion message: the load blocks until the response data returns. The four PCIe traversals of the work-queue-driven path collapse into on-chip-bus crossings roughly an order of magnitude smaller. The trade-off this path pays is that only a subset of the protocol surface is admissible (small loads and stores, plus atomics on the same path; bulk transfers and asynchronous operations stay on the work-queue-driven path).

The load/store path does not replace the work-queue path; it *complements* it. The two cover disjoint operating points: load/store wins where the operation is short and the latency budget is tight; the work-queue path wins where the operation is long and the throughput budget dominates. A NIC that supports only one asks the application to push the wrong shape of operation through the wrong abstraction.

The fourth move: make ordering opt-in. The three moves above form a chain — each enables the next — that pulls the NIC out from behind PCIe. A fourth move is not part of that chain but rides on the per-application objects provisioned by Move 1. RoCE RC enforces strict in-order delivery on every operation, paying the gating cost regardless of need. UB exposes ordering as four orthogonal axes the application opts into per channel and per operation: the service mode controls how aggressively packets may reorder on the wire (relaxed admits multi-path spreading; strict forces single-path delivery); the execution-order tag controls whether the issue stage may emit the next operation before the previous one commits; the fence is an explicit application-level barrier; and the completion-order bit controls whether completions retire in arrival or original-issue order. A pure-fanout broadcast bypasses every gating stage; a barrier-after-collective pays exactly the gating its workload needs.

The gating logic that implements the four axes indexes per-Jetty counters — counters that Move 1's Jetty object already provisions for its own bookkeeping. Opt-in ordering therefore costs zero added pipeline cycles on operations whose mode bypasses gating; the cost is paid only where the application chooses to pay it. A second, less obvious argument is that strict total order is not only expensive on the fast path but fragile under failure: a NIC that has promised in-order delivery on every in-flight operation cannot retire any completion if any earlier operation has stalled, so a single slow path becomes a head-of-line

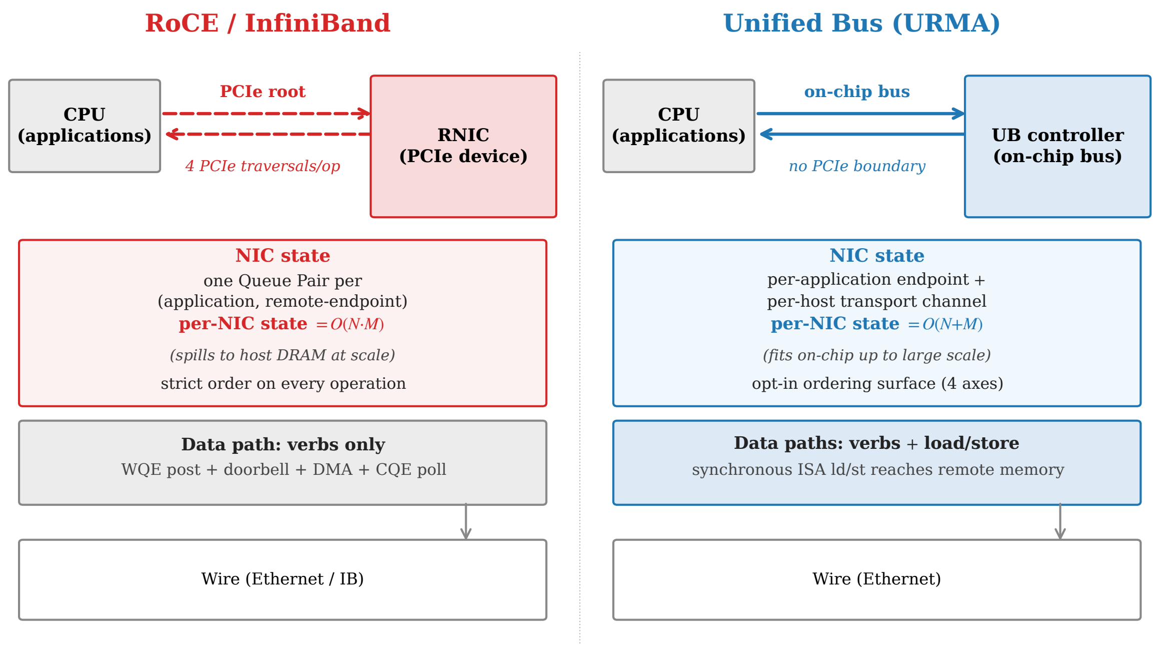


Figure 2: Architectural comparison. RoCE puts the NIC behind PCIe; it holds one Queue Pair per (application, remote-endpoint) pair, enforces strict order on every operation, and admits only the verb-driven data path. Unified Bus puts the controller on the on-chip bus; it holds per-application endpoint state separately from per-remote-host transport state, exposes ordering as an opt-in surface, and admits CPU load/store as a synchronous data path alongside the verb path.

block for every application sharing the channel. The opt-in surface lets workloads request the gating they need without imposing it on the workloads that do not.

2.5 The OpenClickNP toolchain

OpenURMA is built on OpenClickNP [22], a clean-room re-implementation of the ClickNP element model [24]. The model is a familiar one: a NIC pipeline is a directed graph of *elements*, each with typed input and output ports, local state, and a per-cycle handler. The toolchain lowers a single source description through three back-ends: a software emulator for unit testing, a cycle-accurate SystemC simulator for performance measurement, and a Vitis HLS back-end that emits synthesisable C++ for Vivado synthesis on the Alveo U50. The same source produces all three artifacts. The relevance to this paper is twofold: the model makes element-level cost (LUTs, pipeline-II, BRAM) directly measurable, and the three back-ends are how OpenURMA produces both the RTL tier and the SystemC tier from one description.

3. Design

This section describes how OpenURMA realises Unified Bus in RTL. We organise the description around the three architectural commitments introduced in §2.4 — bounded state, opt-in ordering, and the load/store data path — followed by the composition that makes the three mutually load-bearing. The element-by-element decomposition, port wirings, and HLS-pragma specifics are deferred to §4; this section keeps the discussion at the level of how the pipeline embodies each architectural choice and what trade-offs that embodiment forces.

3.1 Realising bounded state

The state model from §2.4 — per-application Jetty objects plus per-host TP Channel objects, with any-to-any addressing through the shared TP Channel pool — maps onto two pipeline regions. The transmit path consults a per-Jetty state table to admit and gate work-queue entries from the local application, then hands each request to a per-host TP Channel scheduler that attaches the right sequence number, picks the multi-path lane, and queues the packet for transmission. The receive path inverts the flow: the per-TP-Channel logic validates and reorders packets by sequence number, then a per-Jetty table maps the destination header to the right local application's receive queue. Two tables, two indices, two scaling regimes.

The implementation cost of the split is one extra header field in every transmitted packet to carry the destination application's identifier, and one extra table lookup on each side of the wire to turn that identifier into a Jetty pointer. The implementation saving — and the architectural payoff — is that nothing in the pipeline binds a transmit context to a receive context. A Jetty can address every remote endpoint it has permission for through the same per-host TP Channel, with no per-pair state created and no per-pair lifecycle to manage. The protocol's $O(N+M)$ scaling is the pipeline's; the table sizes follow.

A consequence of bounded state is that the entire connection working set — per-Jetty state plus per-host TP-Channel state — fits in on-chip BRAM at scales that would force RoCE to host DRAM. At

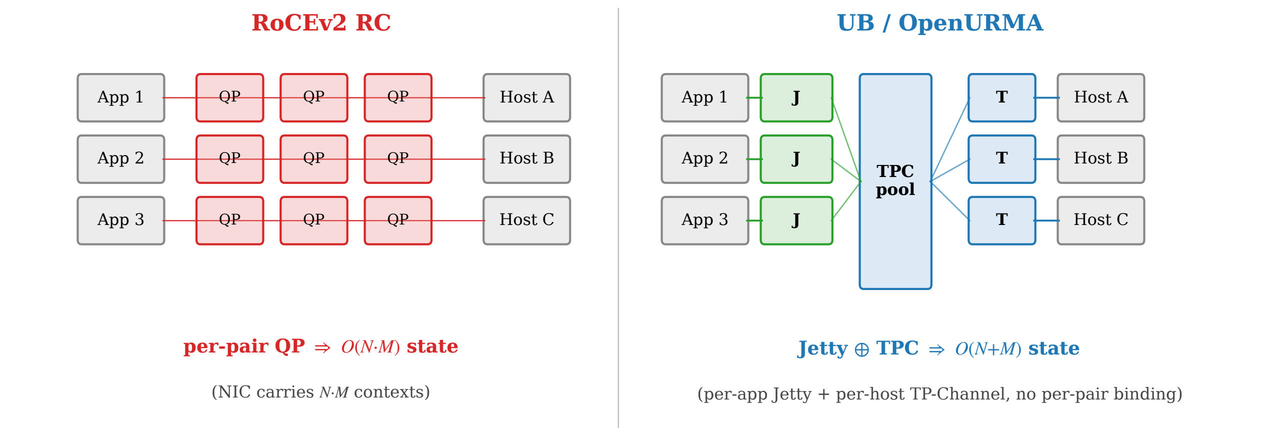


Figure 3: State models compared. RoCE binds one Queue Pair to every (application, remote-host) pair, so per-NIC state grows as $O(N \cdot M)$. UB decouples per-application state (Jetty) from per-host transport state (TP Channel); any Jetty addresses any remote endpoint through the shared TP Channel pool, and per-NIC state grows as $O(N+M)$.

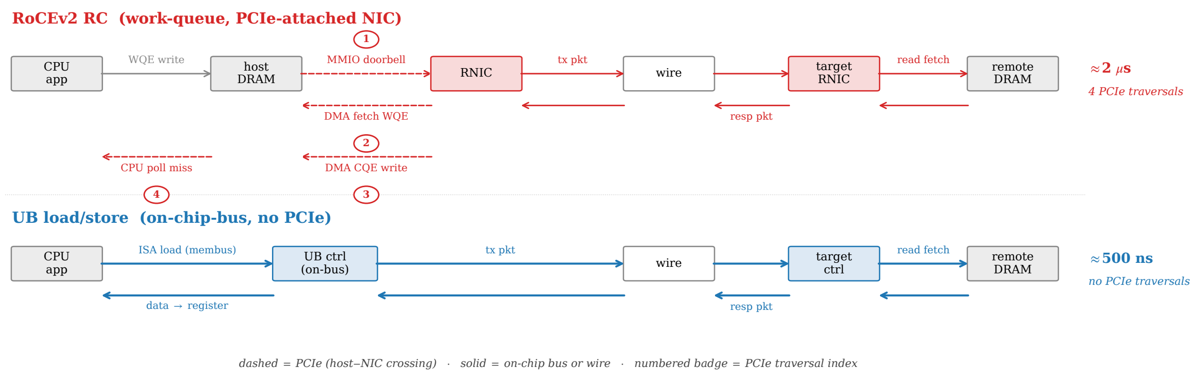


Figure 4: Per-operation data path for a small synchronous read. The traditional work-queue-driven path (top) traverses four PCIe crossings — doorbell over MMIO, work-queue-entry DMA fetch, completion-entry DMA write, and a CPU poll-miss across PCIe coherence. The Unified Bus load/store path (bottom) replaces all four with on-chip-bus crossings: a CPU load instruction reaches the controller, the controller emits the wire packet, the response arrives, and the data lands directly in the CPU register that issued the load.

$(N, M)=(1024, 1024)$, OpenURMA's two-table working set is ∼110 KB; the same workload's RoCE QP table is ∼537 MB. We return to this measurement in §10.3.

3.2 Realising opt-in ordering

The four-axis ordering surface from §2.4 becomes four pipeline structures. Each axis decides at one well-defined stage whether the operation in flight may proceed or must wait; together they form a gating mesh that any operation traverses, but the mesh adds zero pipeline cycles to operations whose mode bypasses all four. The trick is what the gating logic indexes against.

The reused-counter argument. Each gating decision in the ordering surface requires the same inputs: a per-application sequence-number counter (for the "do prior operations from this application still need to complete?" check), a per-channel outstanding-window counter (for the "can the wire absorb another operation?" check), and a per-application fence latch. These three counters are already required by the state model in §3.1 — the per-Jetty table holds them either way, because the spec defines the relevant sequence numbers as per-Jetty objects. The ordering surface reads from the same counters the state model writes to; it does not add a parallel bookkeeping structure. This is what makes opt-in ordering cheap: the per-Initiator machinery the four modes select among is provisioned by the layer split, not added on top of it.

Two reorder buffers, not one. A subtler choice is that the pipeline carries *two* reorder buffers serving disjoint correctness contracts (Figure 6). One acts at the transport layer on packet sequence numbers and delivers byte-correct flits to the upper layer regardless of any application's ordering choice; this is what lets a single TP Chan-

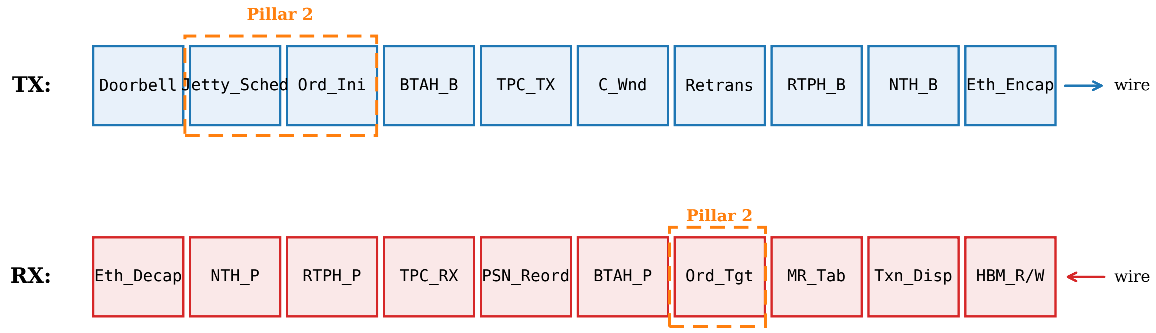


Figure 5: OpenURMA's NIC pipeline, organised by protocol layer. The transaction layer (top half) handles application-level concerns — Jetty scheduling, ordering, memory-region checks, atomic operations, completion generation. The transport layer (bottom half) handles per-host concerns — packet sequence numbers, retransmission, congestion control, multi-path. The two layers communicate through flit-typed boundaries; the load/store data path (§3.3) bypasses the transport layer entirely.

nel multiplex traffic from multiple applications without one application's wire-byte gaps stalling another's. The other acts at the transaction layer on per-application sequence numbers and delivers in-application-order completions regardless of wire-byte order; this is what lets the transmit path spread an application's packets across multiple network paths without breaking the application's view that completions arrive in issue order.

Collapsing the two into one couples the contracts: every multi-path-induced packet gap would block the completion stream; every per-application dependency stall would back-pressure the wire. The separation is what makes the unordered service mode correctness-safe at the same time as the strict mode, and it is what makes multi-path spreading a first-class feature rather than a correctness hazard. We return to this design choice in §12 where we contrast it with the per-transaction bitmap-tracked alternatives that UEC [40] and MP-RDMA [31] take.

3.3 Realising the load/store path

The load/store data path is the most architecturally distinctive part of OpenURMA, because it is the first part where "put the NIC on the on-chip bus" becomes "what does the NIC pipeline actually look like." The work-queue-driven path described above carries ten pipeline stages cold (work-queue scheduling, ordering, header build, transport accounting, congestion check, multi-path lane selection, retransmit-buffer write, framing, transmit). For a small synchronous operation, every one of those stages is overhead the abstraction does not need.

The load/store path drops them. A CPU `ld` or `st` instruction to the address aperture owned by the controller emerges at a single stage that builds the transaction header, attaches a flag declaring transport-layer bypass, builds the network header, and frames the packet for the wire — five stages cold total. No work-queue entry is written, no completion entry is written, no sequence number is allocated (the bypass flag tells the receiving NIC to skip its transport-layer state machine), no retransmit buffer slot is held (the protocol falls back to the CPU's load re-issue on timeout, which is cheaper than maintaining transport-layer reliability for an operation that takes the synchronous path anyway). The trade-off is the one stated in §2.4: the path admits only operations whose semantics survive transport bypass — small loads, stores, and atomics on the same memory — and the controller must be on the same on-chip-bus segment as the CPU. Bulk transfers and async operations stay on the work-queue-driven path because their latency budget tolerates the ten stages.

3.4 Hardware fanout: target-side dispatch

A second consequence of bounded state is that the target-side demultiplexing logic — the step that decides which local application receives an incoming SEND — can live in hardware rather than in a CPU event loop. RoCE handles this in software: the target NIC writes a completion to a shared queue, the application event loop polls, and the dispatch happens at CPU speed. UB defines a *Jetty Group* construct in which a single externally visible identifier represents a set of member Jetties, and the receiving NIC dispatches incoming SEND operations to one of the members in hardware under one of three policies (hashing on an Initiator-supplied key, round-robin, or load-balancing by receive-queue depth) (Figure 7).

The architectural payoff is that the target CPU is not on the SEND fast path. The trade-off is one extra pipeline element on the RX path between transaction dispatch and the per-Jetty receive logic, with per-group state on the order of $\sim$80 B for a 8-member group. We measure the end-to-end saving in §8.10: $\sim$200 ns per SEND when the fanout is small, growing to $\sim$1200 ns once the RoCE side begins paying for QP-state spill on top of the CPU dispatch.

3.5 How the three commitments compose

The implementation makes visible a coupling the introduction asserted but could not justify in detail there: each of UB's three architectural commitments depends on the others being present.

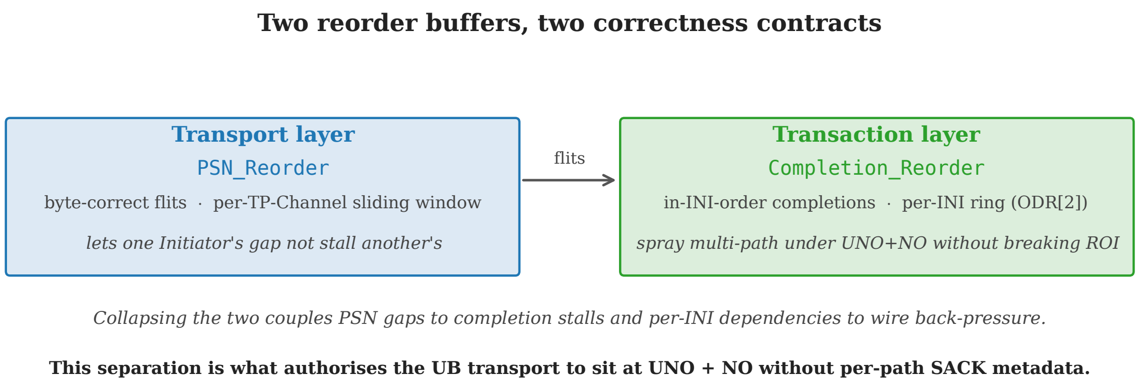


Figure 6: The pipeline carries two reorder buffers serving disjoint correctness contracts. Packet-sequence reordering at the transport layer delivers byte-correct flits regardless of application ordering; transaction-sequence reordering at the transaction layer delivers in-issue-order completions regardless of wire-byte order. The separation is what makes multi-path spreading and opt-in ordering composable.

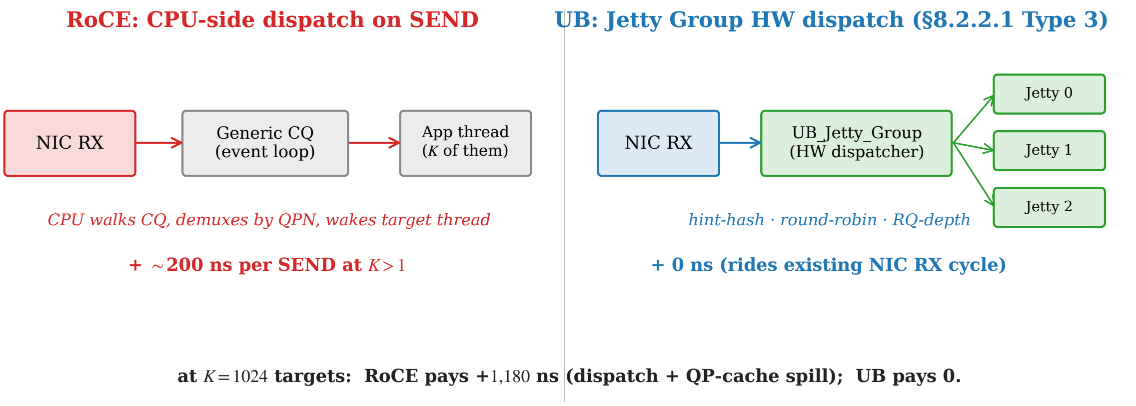


Figure 7: Target-side SEND dispatch. RoCE demultiplexes through a shared completion queue plus an application event loop; UB's Jetty Group performs the dispatch in hardware on the membus crossing already paid for by NIC RX.

The on-chip controller depends on bounded state. The load/store path of §3.3 lives on the on-chip bus because it does not need DRAM access to look up the per-application state required to authorise the operation; the bounded state of §3.1 keeps that state in on-chip BRAM. If state spilled to host DRAM, the controller would have to issue an external memory access on every load/store, which would put it back in PCIe-latency territory and defeat the point of the on-bus placement.

Opt-in ordering depends on the layer split. The gating logic of §3.2 reads counters the per-Jetty table already provides; the gating logic costs roughly 13 KLUT (10.6% of the design's LUT budget) and *zero* added pipeline cycles on operations whose mode bypasses gating. If the state model had not split per-application from per-host state — if the design had remained QP-centric — the ordering surface would have to add the per-application counters from scratch, and the cost would compound on every operation regardless of need.

Multi-path spreading depends on the two-reorder split. The unordered service mode authorises a transmit path to spray packets across multiple wire paths for bandwidth and resilience. The split between transport-layer and transaction-layer reordering (§3.2) is what makes that spreading correctness-safe without per-transaction SACK metadata: an unordered application receives unordered completions; an ordered application sharing the same wire receives ordered completions; neither stalls the other. Collapsing the buffers would force one contract onto both.

These dependencies are why we say the three commitments are three faces of one architectural commitment. The next two sections show what that commitment costs (§4) and what it saves (§6 onwards).

4. Implementation

This section opens the implementation up to the level of detail the design discussion deliberately deferred. We describe the pipeline decomposition, the two new pipeline elements introduced for the load/store path and target-side fanout, the toolchain extensions, the multi-tier simulation infrastructure (SystemC and gem5), and the timing-closure sweep that produced the 322 MHz post-route result. Throughout, we describe what each piece *does* rather than what its source file is named; the implementation is open and source-level identifiers are what they are at the time of writing, but the protocol responsibilities the implementation maps onto are stable.

OpenURMA is roughly 3.5 KLOC of element-level source spread across 39 pipeline elements implementing the transport and transaction layers. The parallel OpenRoCE baseline (RoCEv2 RC) is roughly 1.3 KLOC across 21 elements. Both stacks sit on the same OpenClickNP toolchain [22], so all measurements compare implementations produced by an identical lowering pipeline.

4.1 Pipeline decomposition

The 39 OpenURMA pipeline elements break into six functional categories. The counts and responsibilities are what the protocol requires.

- **10 header parsers and builders** handle the four layered headers the spec defines — Ethernet framing, a network-transport header, a reliable-transport header, an unreliable-transport variant, and a base-transaction header — as matched parser/builder pairs.
- **9 transport-layer elements** implement the per-host TP Channel pipelines on both ends, the retransmission buffer, the packet-sequence reorder buffer, the two acknowledgement-generation elements (one for the lower layer's per-packet acks, one for the upper layer's transaction-level acks), the congestion-control window, the congestion-echo feedback element, and the multi-path transport-protocol-group dispatcher.
- **4 ordering elements** implement the gating surface introduced in §3.2: the Jetty scheduler at admission, two order trackers (one each at the initiator and the target, because the gating responsibility shifts between endpoints depending on the service mode), and a transaction-layer completion reorder buffer.
- **7 data-path elements** handle on-NIC memory reads and writes, atomic operations, per-Jetty receive logic, the new target-side dispatcher (§4.2), the transaction dispatch-by-opcode router, and the completion generator.
- **3 state tables** hold the memory-region permissions, the per-Jetty state, and the per-TP-Channel state.
- **6 glue elements** provide the doorbell, dispatch multiplexers, completion notification tees, completion-queue streamers, and the retransmission RTO timer.

Three pipeline-decomposition choices deserve note. The initiator-side and target-side order trackers are distinct elements because the gating responsibility moves between endpoints depending on the service mode (the relaxed mode places it at the target; the strict mode places it at the initiator). The transaction-layer completion reorder buffer is a distinct element rather than a flag on the per-Jetty receive logic because the in-order-versus-arrival-order choice is per-operation and requires a per-application sliding window, not a per-Jetty mode bit. The packet-sequence reorder buffer is distinct from the transaction-layer completion reorder buffer for the two-buffer reason discussed in §3.2: the two serve disjoint correctness contracts on disjoint sequence-number spaces.

The transmit path threads through ten pipeline stages cold for the work-queue-driven path: doorbell admission, Jetty scheduling, initiator-side ordering check, base-transaction-header build, TP-Channel transmit logic, congestion-window check, retransmission buffer write, reliable-transport-header build, network-transport-header build, and Ethernet encapsulation. The receive path inverts the sequence. The full topology is in Figure 5.

4.2 Two new pipeline elements

Two pipeline elements are not derivable from existing OpenClickNP templates and required new design.

The load/store transport-bypass engine. The five-stage transport-bypass pipeline introduced in §3.3 is implemented as a single new element threaded into a separate topology variant alongside the default work-queue-driven topology. The transmit handler accepts a load or store doorbell request from the on-chip-bus side, allocates a one-shot transaction context (no per-channel sequence-number allocation, no retransmission-buffer slot), stamps a transport-bypass flag in the next-layer-protocol field so the receiving NIC skips its transport state machine, and frames the packet. Posting a single load request, the first wire flit emerges at **8 cycles** $\approx$25 **ns at 322 MHz** — 17 cycles ($\approx$53 ns) below the work-queue-driven path on identical infrastructure. The saving is the protocol-layer instantiation of spec-mandated transport bypass; it is not microarchitectural.

The target-side hardware dispatcher. The target-side hardware dispatcher introduced in §3.4 sits between transaction dispatch and the per-Jetty receive logic on the RX path. When an incoming SEND addresses a registered Jetty-Group identifier, the element rewrites the extension flit's destination field to point at one of up to eight member Jetties according to a per-group policy: *hint-hash* (the member index is the application-supplied hint modulo the member count, which lets the initiator steer requests by an opaque key as long as the application chooses the key consistently), *round-robin* (a cursor advances per SEND), or *receive-queue-depth load balancing* (the member with the shallowest pending receive-queue depth wins). SENDs to unregistered identifiers pass through unchanged, so the element is opt-in per group. Per-group state is on the order of $\sim$80 B for a maximum-membership (8-member) group: one identifier and two counters per member, plus a small header.

4.3 Retransmission and congestion control

The retransmission buffer is a 64-slot in-flight ring per TP-Channel with both Go-Back-N and selective-retransmit modes. The selective-retransmit path uses the spec-defined selective-ack response opcode plus a per-PSN bitmap in the acknowledgement header. The retransmit decision is driven either by a transaction-layer acknowledgement that explicitly NAKs PSNs (selective) or by the RTO timer firing (Go-Back-N). Each slot captures one metadata-and-extension flit pair, which is sufficient for control-plane work requests and for $\leq$8-byte Writes. *Multi-flit retransmission* — extending the slot to a payload-flit list so a dropped multi-flit Write can be replayed in full — is implemented in the loss-free Write data path (§6) but not yet in the retransmit buffer; we flag this in §11.

The congestion-control loop combines a host-side switch model with a per-channel additive-increase / multiplicative-decrease window. The switch is a SystemC and software-emulator element that maintains a queue with low/high watermarks and stamps the forward explicit congestion-notification bit on packets when occupancy crosses the high watermark. The Initiator-side the congestion-window element listens for marked acknowledgements and applies a multiplicative decrease; absent marks, it additively increases per RTT. The closed loop is verified in the closed-loop test: 200 work requests posted, 9 of the 25 packets that traverse the switch are marked, and the sender's window drops from 65,536 B to 4,096 B in response.

4.4 Toolchain extensions

Six pipeline elements needed pipeline-initiation-interval guidance to close 322 MHz; the back-end previously hard-coded II=1, which over-pipelines elements whose data dependencies cannot be unrolled aggressively. We added two pragma blocks to the element-source grammar: one that lets an element declare its initiation interval (II) explicitly, and one that lets an element forward arbitrary HLS pragmas verbatim. Both extensions are upstream in OpenClickNP and reused unchanged by both stacks. They are the only modifications we made to the framework.

4.5 Three back-ends from one source

OpenClickNP lowers each `.clnp` element through three back-ends. The first is a software emulator that wraps each element's per-cycle handler in a a thread connected to bounded SPSC FIFOs; this is the back-end the correctness tests of §6 run against. The second is a cycle-accurate SystemC simulator with a 1 ns clock period matching the 322 MHz synthesis target, so a SystemC cycle count is directly comparable to post-route Vivado timing; this is the back-end the microbenchmarks of §7 use. The third is a Vitis HLS back-end that emits synthesisable C++, which Vivado then synthesises and places-and-routes to a chosen Xilinx part; this is what produces the LUT, BRAM, and timing numbers of §10. All three back-ends consume the same `.clnp` source, so an element that closes timing in HLS corresponds exactly to the element the SW emulator tested and the SystemC simulator benchmarked.

4.6 Two-node SystemC simulator

The two-node simulator links each NIC stack as a static SystemC library and wires two instances of each side through a SystemC EtherLink module with configurable bandwidth and propagation delay. Each endpoint is a full system: a CPU host model that constructs work-queue entries and posts doorbells, the NIC's transmit and receive pipelines, a DDR4 DRAM model for both work-queue-side and data-side accesses, a two-level write-back / write-through / uncached cache hierarchy for the host CPU, and a completion-queue write-back path. The harness produces a 388-configuration sweep covering all six verb categories, four cache policies, payload sizes from 8 B to 4 KB, and link delays from 100 ns to 10 μs.

4.7 gem5 full-system scaffold

For the CPU-to-remote-DRAM end-to-end claim, we integrate the SystemC NIC into gem5 as a memory-mapped device. The integration required three load-bearing changes against the current gem5 release.

SystemC ABI rebuild. Gem5 ships its own embedded SystemC headers; the upstream Accellera SystemC build the OpenClickNP cycle-accurate back-end uses is ABI-incompatible with them. We rebuild the NIC SystemC libraries against gem5's headers, with a separate output target, and point the gem5 build script there.

Port wiring and address-range partition. Gem5's memory-bus crossbar cannot route overlapping address ranges, so the NIC's address aperture must be carved out of the system memory range that the standard configurations use by default. The controller's initialisation entry point must also explicitly call the crossbar's range-change hook so the bus's address-range latch settles before gem5's binary loader issues its first write through the port proxy.

Dual-node configuration. A dual-node configuration brings up two ARM CPU instances with the SystemC EtherLink between their controllers, runs a small set of SE-mode binaries on each that exercise the NIC's address aperture through ordinary load and store instructions, and pre-maps the NIC aperture into each process's page table after binary load so the CPU's loads and stores reach the controller without going through gem5's full virtual-memory subsystem. The current scaffold validates the end-to-end CPU-to-remote-controller path with SE-mode workloads; full-system Linux integration is follow-on work.

4.8 Timing-closure sweep

The initial Vitis-HLS pass on the 38 work-queue-driven elements left 8 elements needing remediation: 5 missed 322 MHz in Vivado P&R and 3 failed at HLS itself before reaching P&R. The mechanical resolution combined the new II and HLS-pragma extensions described above with a few targeted algorithmic redesigns, summarised by element function in Table 1.

Element function	Before → After (ns WNS)	Action
Atomic operations	$-1.871 \rightarrow +0.298$	II=4 + memory partition
Completion reorder	HLS-stuck → $+0.672$	head-pointer ring + II=2
Initiator order tracker	$-5.382 \rightarrow +0.318$	II=2
Jetty scheduler	HLS-aborted → $+0.546$	II=2
Target order tracker	$-2.25 \rightarrow +0.101$	II=2 + drain re-order
On-NIC memory write	$-0.628 \rightarrow +0.628$	memory partition
Memory-region table	failing → $+0.729$	shrink table to 64 entries
On-NIC memory read	$-0.449 \rightarrow +0.282$	word-sized array

Table 1: Timing-closure remediation, by element function.

One instructive failure. The most instructive case is the on-NIC memory-read element, which originally backed its 64 KB read path with a byte-sized array plus cyclic memory partition for parallel byte-bank access. HLS estimated 490 MHz $F_{\max}$; Vivado P&R reported -0.449 ns WNS. The worst path ran from a register inside the per-bank index computation into the BRAM address port of the partitioned array: 8 LUT levels of barrel-shift mux to compute the per-bank index, plus 2.7 ns of routing delay (76% of the period). No combination of II or partition factor moved the slack; the wide muxing for the 8 banks bloated routing without unblocking timing. The fix was algorithmic: replace the byte-sized array with an 8-byte-word-sized array indexed by a shifted offset. Each 8-byte-aligned read becomes a single BRAM word access; the byte-bank mux is gone; the memory partition pragma is no longer needed. WNS closes at $+0.282$ ns. The same pattern was applied to the OpenRoCE on-NIC memory-read element. The lesson is that HLS's per-element timing estimate is a useful but optimistic predictor; routing-bound paths only show in Vivado P&R, and the right resolution may be at the data-layout level rather than the pragma level.

4.9 Test coverage

The 16 SW-emu integration tests (enumerated in §6) cover correctness across the spec surface, the wire format, the closed congestion-control loop, the multi-flit Write payload path, the fused-acknowledgement fusion, and the full atomic opcode set. A sustained-throughput sanity check posts 50,000 work requests through the same harness. The SystemC back-end runs a cycle-accurate latency benchmark that drives requests through the entire transmit-to-wire-to-receive path and reports first-flit latency, round-trip latency, and sustained throughput. The HLS synthesis sweep and Vivado P&R sweep are scripted for one-button reproducibility.

5. Evaluation Methodology

Because no FPGA is available for in-silicon experiments, we evaluate the design through three independent paths, each answering a different class of question:

- **SW-emulator (functional correctness).** Each `.clnp` element compiles to a C++ kernel that runs in a a thread with bounded SPSC FIFOs between elements. Tests post WRs into the pipeline's input stream, drain responses, and assert spec-defined behaviour.
- **SystemC (cycle-accurate).** The same elements compile to per-element SystemC modules, one a 1-ns wait per loop iteration so that 1 ns of simulation time corresponds to one cycle at the 322 MHz target. We measure cycle counts directly and convert to wall-clock by multiplying by the post-route clock period (3.106 ns).
- **Vivado synth+place+route (real RTL).** Each element's HLS-synthesised Verilog goes through a full Vivado out-of-context flow targeting the Alveo U50 (the Alveo U50 part). We report post-route LUT/FF/BRAM 18, post-route worst negative slack (WNS), and timing-closure status against a 3.106 ns clock period.

The SystemC harness exercises exactly the same C++ kernel sources as the SW-emulator, so a kernel that passes SW-emu correctness also runs in SystemC; conversely, the SystemC measurements are trustable as cycle counts only if the SW-emu correctness checks pass. Vivado synth runs on the HLS-emitted Verilog from the same `.clnp` sources.

Every number in the rest of the paper is reproducible from the released evaluation harness, which drives correctness, Vivado P&R, and the SystemC microbenchmark sweep end to end and writes all results out as CSV.

6. Functional correctness

Seventeen SW-emu tests cover the spec surface and the framework internals; all pass on the current source tree.

- **Initiator-side ordering** — a strict-order operation is gated behind a relaxed-order operation; the pipeline holds the strict op at the initiator until the relaxed one commits.
- **Target-side ordering** — the target serialises strict-order operations at execution time, deferring them behind missing prior operations.
- **Fused acknowledgement** — the lower-layer ack carries the upper-layer ack on the same flit, saving one wire packet per response under the fused-ack service mode.
- **Fence** — a fenced operation waits until all prior reads and atomics have responded before its own emission.
- **Completion ordering** — with the in-issue-order bit set, completions are retired in original-issue order; with it clear, they are retired in arrival order.
- **Unordered Send wire path**.
- **Mixed-mode operations** in the same send queue.
- **Multi-initiator parallelism** — per-initiator gating: a strict-order operation on one initiator emerges immediately while a strict-order operation on a different initiator is gated behind its own dependency.
- **Head-of-line-blocking isolation** — 8 unordered operations from 4 initiators all emerge despite a stalled strict-order operation on a separate initiator.
- **On-NIC memory read/write data integrity**.

- **Full atomic opcode set** — swap, store, load, fetch-and-add, fetch-and-subtract, fetch-and-and, fetch-and-or, fetch-and-xor; each verified to return the pre-modification value in the response and to leave the on-NIC memory word in the algebraically correct post-state. Compare-and-swap is covered by the mixed-mode test.
- **Transmit-to-wire round trip** with all UB header fields preserved.
- **Sustained-throughput sanity** (50,000 work requests).
- **Wire-format encoder** against the spec bit layout.
- **End-to-end congestion-control loop** — 200 work requests posted; 9 of the 25 packets that traverse the modeled switch are congestion-marked; the sender's congestion window backs off from 65,536 B to 4,096 B.
- **Target-side hardware dispatch** — the dispatcher routes incoming SENDs to one of N member Jetties under three policies (hash on an application-supplied hint, round-robin, receive-queue-depth load balancing). The test exercises all three on a 4-member group plus the unregistered-identifier bypass; the end-to-end consequence is quantified in §8.10.
- **Multi-flit Write payload path** — 8/32/64/128/256-byte writes through Ethernet-encapsulation streaming, decapsulation reassembly, and the on-NIC memory-write payload byte stream.

What the SW-emu suite does *not* cover. The eval as it stands exercises every implemented mode and opcode on the loss-free path. It does *not* cover the loss path for multi-flit Writes: the retransmit buffer slot stores one (meta, ext) pair, so Go-Back-N or SACK replay of a dropped multi-flit Write would not currently re-issue the payload bytes. Enabling that requires extending the retrans slot to a payload-flit list (a follow-on work item, §11). All single-flit retransmit paths (control-plane WRs, $\leq$ 8 B Writes, Reads, Atomics) are exercised by the C-AQM closed-loop test through Cong_Echo, but explicit drop-injection tests remain follow-on work.

7. Cycle-accurate microbenchmarks

We instrument the TX pipeline (the harder of the two paths to reason about, since it is where the ordering surface sits) under several SystemC scenarios. Each scenario runs as one SystemC microbenchmark invocation and writes one CSV row per parameter point.

7.1 Per-stage latency breakdown

A single Write WR (ROL+NO) drives the pipeline cold. Tap monitors inserted between every adjacent stage record the first-flit arrival time. Figure 8a shows the per-stage contribution.

The slowest stages are the Jetty scheduler (5 cycles — $II{=}2$ plus the per-Initiator round-robin scan) and Ethernet encapsulation (11 cycles — byte-stream-rate, not flit-rate). The initiator-side order tracker costs 1 cycle on the unblocked path: the gating logic adds combinational depth (covered in §10) but no extra pipeline cycles when no prior dependencies are outstanding. Total cold-path TX latency: 24 cycles $\approx$ 75 ns at 322 MHz.

7.2 Per-mode TX latency

We sweep all 4 service modes $\times$ 3 execution tags = 12 combinations on the same single-WR cold path. *Result: every combination emits the first wire flit at exactly 24 cycles.* The per-mode penalty is therefore not in pipeline-stage count but in the combinational logic depth of each kernel — and that is what shows up in the Vivado area report (§10), not in the SystemC cycle count. This is the design's intended behaviour: opt-in ordering does not add pipeline cycles when not used.

7.3 Sustained throughput per service mode

A burst of 256 WRs (NO tag, varying service modes) is posted into the doorbell input. Steady-state throughput is calculated as $(N{-}1)/(\Delta t)$.

Service mode	Steady WR/μs
OpenURMA: ROI / ROT / ROL / UNO	**150.36**
OpenRoCE RC (matched microbench)	53.62

All four OpenURMA service modes share a single transmit pipeline (the unordered mode bypasses sequence-number allocation within the TP-Channel transmit element but still traverses the same kernel chain), and the throughput floor is set by the slowest pipeline initiation interval in the chain ($II{=}2$ in the Jetty scheduler, the initiator-side order tracker, the completion reorder buffer, and the target-side order tracker; $II{=}4$ in the atomic-operation element). Hence the per-mode rate is identical at 150.36 WR/μs. The OpenRoCE baseline runs the matched-shape microbenchmark on identical infrastructure and measures 53.62 WR/μs — **2.80$\times$ slower** — because RC's per-QP sequence-number allocation introduces stricter inter-operation dependencies in the QP transmit kernel. The 1000-WR burst-saturation regime reaches 159.46 WR/μs on OpenURMA and 53.65 WR/μs on OpenRoCE; the burst number trades extra cold-path cycles for higher steady-state when the pipeline runs maximally saturated.

7.4 Ordering surface: Fence and serialised-order gating cost

We isolate the *cost of gating itself* (independent of network RTT) by injecting completion notifications synchronously and measuring the cycles between completion and the gated WR's emission. Figure 8b plots both costs.

- **Fence (Jetty_Sched, §7.3.2.2 note).** Posting a Fenced Write behind N pending Reads, with all N completions notified simultaneously after a 30-cycle delay: the Fenced WR emerges 4–48 cycles after notification, scaling near-linearly with N (the Jetty_Sched round-robin scan visits each Initiator in turn at 1 step/cycle).
- **SO (OrderTracker_Initiator, §7.3.3.2).** Posting an SO behind N outstanding ROs, with all N completions notified simultaneously: SO emerges 7–38 cycles

after notification. The cost is bounded by ord_ini's 16-position round-robin cursor.

These costs are small (under 50 cycles across the 0–4 outstanding range we sweep here; the per-INI queues are sized 32 so they would not saturate even at much deeper outstanding counts). Crucially, they are paid only when gating is requested; an application that uses NO+UNO sees neither.

7.5 Cross-INI head-of-line isolation

We force one Initiator (rc_id 0xA) into a permanent ROI+SO stall by emitting an RO and then a gated SO whose RO completion is never notified. We then post 8 UNO+NO WRs from four different Initiators (0xB0..0xB3). Figure 8c shows that all 8 stream-B WRs emerge at the wire within 78 cycles of post — they bypass the gating elements completely. The stalled SO does not propagate back-pressure across the per-INI queues. This is the key ordering-surface guarantee that is impossible under RC: head-of-line on one connection does not block another.

7.6 Throughput vs payload size

Figure 8d sweeps payload from 8 B to 4 KB. The per-WR rate is essentially constant at $\approx$141 WR/μs, confirming that the pipeline is metadata-flit-rate-limited at small-to-medium payloads — exactly the regime that matters for AI collective control packets and HPC small-message all-to-all. At 4 KB payload (129 wire flits per WR), the same WR rate translates to a wire-byte rate of $\approx$4.6 Tb/s — well above the U50's single-port CMAC line rate, indicating that bandwidth is not the binding constraint at any payload size in this regime.

7.7 Load/store transport-bypass cold path

The microbenchmarks above exercise the work-queue-driven path, which traverses ten pipeline stages cold (doorbell admission, Jetty scheduling, initiator-side ordering, base-transaction header build, TP-Channel transmit, congestion-window check, retransmission-buffer write, reliable-transport header build, network-transport header build, and Ethernet encapsulation). The load/store path replaces all ten with five: a CPU load or store reaches an on-chip-bus-attached controller, the controller flags the request for transport bypass, and the packet emerges through the header builder and encapsulator. The transport layer is omitted entirely (no sequence-number allocation, no retransmission slot, no transport-layer header). Posting a single load request, the first wire flit emerges at **8 cycles $\approx$ 25 ns at 322 MHz** — 17 cycles ($\approx$53 ns) below the work-queue-driven cold path on identical infrastructure. The saving is structural, not microarchitectural; it removes the transport-layer state machine from the NIC budget for the workloads that opt in.

Why load/store, specifically, admits transport bypass. The restriction is structural. Atomic operations need remote serialisation (a second fetch-and-add must observe the first's commit), and the transport layer is what provides that order. Read and Write route through the work-queue path to preserve completion-queue semantics for the asynchronous verb surface. Load and store, by contrast, carry no inter-operation ordering contract at the application level — the CPU's load-use dependency chain already provides the order the transport layer would otherwise enforce — so the transport layer is redundant and elides.

Is 8 cycles a substrate floor? Yes: each of the five pipeline stages already runs at II=1, and the remaining 3 cycles are state-machine setup at the boundaries. Collapsing two stages would push the critical path through two parsers in one cycle and miss the 3.106 ns period — the same routing-density wall the on-NIC memory-read story (§4) formalises. An ASIC at 322 MHz is still 8-cycle-floored; an ASIC at 1 GHz yields 8 ns absolute, and tighter cell delays let adjacent stages collapse to 5–6 cycles. He's [8] $\sim$100 ns end-to-end UB ASIC number is consistent with this budget ($\sim$5–8 ns each NIC + link + DRAM).

8. Two-node end-to-end evaluation

The microbenchmarks in §7 measure the NIC pipeline alone. The architectural argument UB attaches to memory-semantic access (He [8], spec §8.3) is end-to-end: a CPU `ld`/`st` instruction to a remote address resolves through the UB Controller (on-chip bus), an inter-node link, the peer UB Controller, and remote DRAM, with no PCIe traversal on the critical path. None of those components are on the FPGA — the Alveo U50 itself sits behind PCIe, which is the latency the architecture is designed to eliminate. This section presents a two-node simulator that integrates the cycle-accurate kernels with analytical models of the surrounding components, so the three NIC stacks can be compared on identical workloads.

8.1 Methodology

The simulator is a SystemC program that links each of the three NIC stacks (UB work-queue, UB load/store, RoCEv2 RC) as a static SystemC library and instantiates two endpoints of each kind. Each NIC transmit/receive cycle count is taken directly from the corresponding cycle-accurate SystemC microbenchmark at 322 MHz: 8 for the UB load/store transport-bypass path, 25 for the UB work-queue-driven path, and 9 for RoCEv2 RC.

Decomposed cost model — full bidirectional accounting. Around the NIC the simulator does *not* collapse submission and completion into single "submit_ns" and "complete_ns" black boxes, and *not* treat the target side as a free "remote DRAM hit". Every host$\leftrightarrow$NIC interaction on *both* sides of the wire is an explicit SystemC component on the critical path. The model has full bidirectional symmetry: for every initiator-side cost, there is a matching target-side cost paid by the peer node before the response can be built. The target side is not

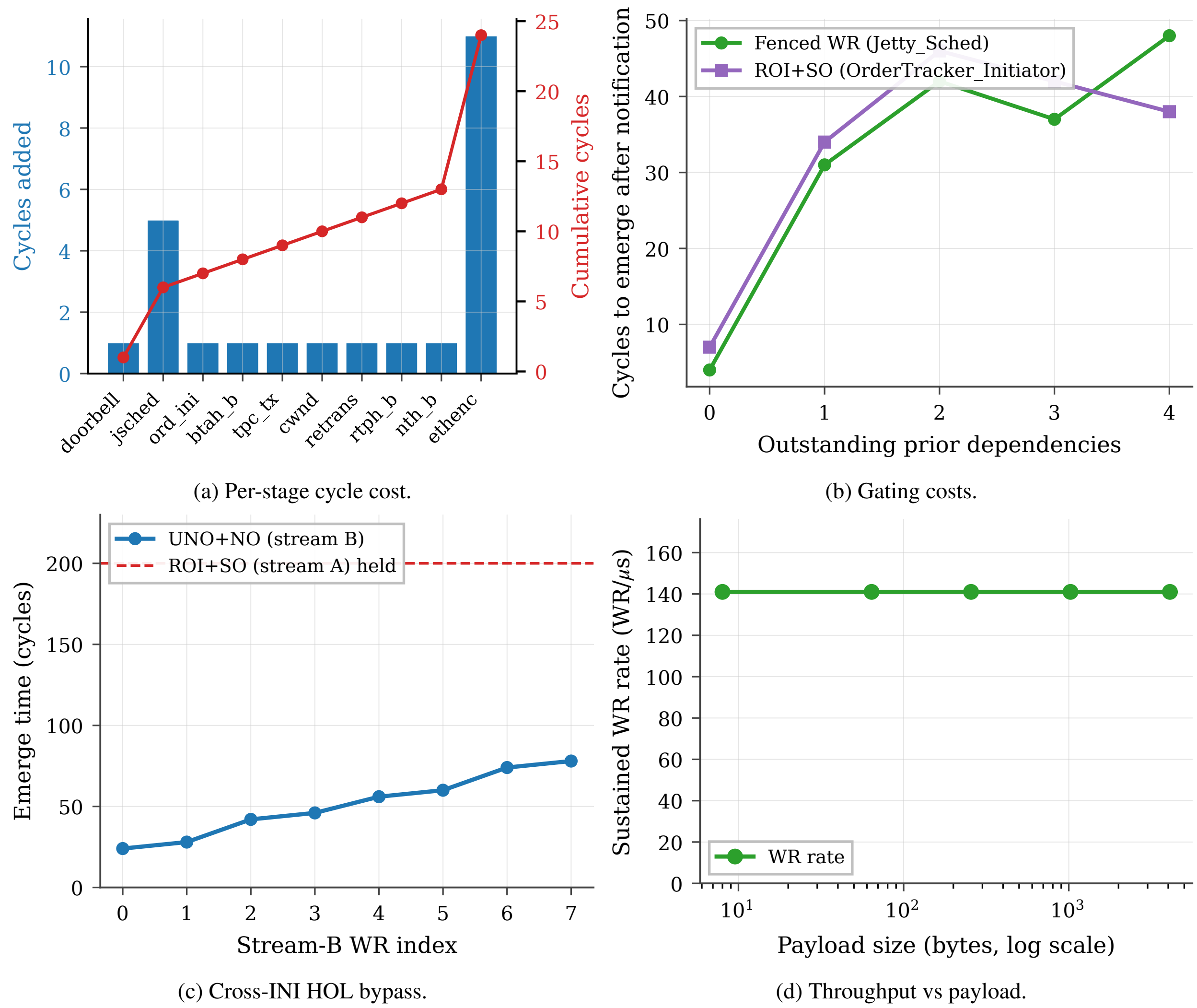


Figure 8: SystemC microbenchmarks. (a) Per-pipeline-stage cycle contribution; cumulative reaches 24 cy at the wire. (b) Cost of Fence and SO gating in cycles after completion notification. (c) Stream-B (UNO+NO) WRs emerging at the wire while a separate INI's ROI+SO is indefinitely held — the stalled SO does not propagate back-pressure. (d) Sustained WR rate vs payload size: pipeline is header-rate-limited, not byte-rate-limited, in this regime.

a passive responder; the target NIC pays its own NIC RX pipeline cycles (the "stack processing" cost), its own NIC↔DRAM transfer (PCIe for RoCE, membus for UB), the DRAM row-hit itself, and its own NIC TX pipeline cycles to build the response packet.

- **Verb-library post overhead** (50 ns) — user-space verb library overhead on the post-send call: function dispatch, send-queue validation, a store-ordering memory barrier, lock acquisition. Zero for the load/store path (the verb IS the ISA instruction; no library call).
- **Work-request construction** (30 ns) — CPU stores building the work-request structure in host memory. Zero for the load/store path (no work-request structure).
- **Doorbell MMIO** (150 ns) — posted PCIe write to the NIC's MMIO BAR. Zero for UB (on-chip bus).
- **Work-queue-entry DMA fetch** (500 ns) — NIC-initiated PCIe DMA read fetching the work-queue entry from host DRAM. Zero for UB and for the RoCE inline-WQE path with WQE $\leq$ 64 B.
- **On-chip-bus traversal** (30 ns) — controller on-chip-bus traversal, used for both submission and completion on the UB stacks.
- **Completion-entry DMA write** (250 ns) — NIC-initiated PCIe DMA write posting the completion entry to host DRAM. RoCE only.
- **Completion-entry poll** (70 ns over PCIe, 5 ns on-chip) — CPU spin-load of the completion record. Zero for the load/store path (synchronous; the load returns directly to a register).
- **Verb-library overhead** (30 ns) — the user-space verb library walks the completion queue, marshals the work-completion structure, and returns to the caller. Zero for the load/store path. Applied symmetrically to UB's work-request path and to RoCE so the software-side cost is not asymmetric between the WR-based stacks.
- **Target-side NIC–DRAM transfer** — this is the cost the "single-node" RDMA latency literature most often skips. The target NIC must move the operand between itself and the target host DRAM *before* (for READ and atomic operations) or *after* (for WRITE and SEND) the local DRAM row access. RoCE pays a PCIe DMA on every operation (500 ns for READ/atomic, 250 ns for

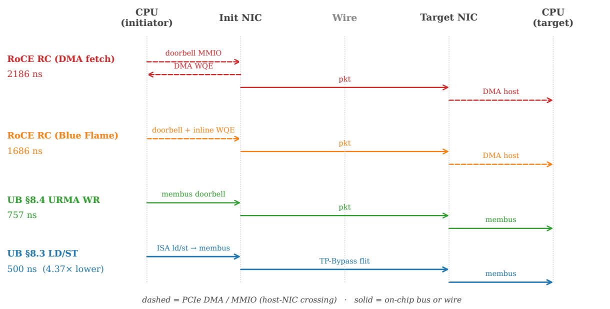


Figure 9: Submission path per stack. RoCE traversals between CPU and NIC (dashed) sit on PCIe; UB carries the same hand-offs over the on-chip bus. On UB §8.3 Load/Store the verb *is* an ISA instruction, and the wire crossing carries a TP-Bypass flit instead of a full RTPH-wrapped packet.

WRITE/SEND); UB pays only the on-chip-bus crossing (30 ns) because the controller is on the on-chip bus on the target as well.

- **Initiator-side response DMA** — for READ-class verbs, the initiator-side payload DMA *after* the response packet arrives (the NIC DMA-writes the returned payload to host DRAM, separately from the completion-entry write). 250 ns for RoCE; zero for UB (the payload returns on-chip).

Figure 9 sketches each stack's round trip — which traversals are PCIe (dashed) and which are on-chip or wire (solid) — and Table 2 consolidates the full per-side decomposition for a 64 B READ. Every row is paid on the critical path before the next phase can start. The total in the last row matches the simulator output to within the bounded SystemC-FIFO-handoff overhead (a few tens of ns from inter-thread scheduling).

Defaults follow published ConnectX-7-class ranges [35, 18]; every value is exposed as a command-line knob so the comparison can be repeated under different parameter assumptions. We model only polling completion — the regime high-performance RDMA uses — and deliberately omit event-driven completion: the latter requires modelling MSI-X delivery + kernel interrupt-handler dispatch + scheduler decisions + thread wakeup, all of which are OS-dependent and cannot be faithfully represented by a single analytical parameter; FastWake [25] characterises that host-side path directly. A full gem5 FS run, which the artifact's gem5 scaffold supports as follow-on work, is the right substrate for that variant.

CPU-side cache hierarchy. A two-level cache (L1, L2) + LLC + local DRAM hierarchy in the simulator is consulted on every §8.3 `ld/st`. Fully-associative LRU replacement, three policies (write-back / write-through / uncacheable), default hit latencies L1: 1 ns, L2: 4 ns, LLC: 12 ns, local DRAM: 70 ns. §8.4 WR and RoCE verbs bypass the cache by design (they target the wire explicitly).

Table 2: Per-side latency decomposition (ns) for a READ verb, 64 B payload, link delay 100 ns, polling completion. Every row on the table is paid on the critical path. "Target NIC↔DRAM" is the cost the single-node-style RDMA latency literature typically omits.

Phase	UB LD/ST	UB URMA	RoCE DMA
Initiator side, pre-wire			
Verb library post	0	50	50
WQE construct	0	30	30
Doorbell MMIO	0	0	150
DMA WQE fetch	0	0	500
Submit (membus)	30	30	0
NIC TX pipeline	25	78	28
Wire forward	100	100	100
Target side, between wire and DRAM			
NIC RX (stack proc.)	25	78	28
Target NIC↔DRAM	**30**	**30**	**500**
Target DRAM row hit	30	30	30
NIC TX (response)	25	78	28
Wire back	100	100	100
Initiator side, post-wire			
NIC RX (response)	25	78	28
Initiator resp DMA	0	0	250
DMA CQE write	0	0	250
Complete (membus)	30	30	0
CQE poll	0	5	70
Verb library poll	0	30	30
Total (modeled)	**420**	**745**	**2172**
Total (measured)	**500**	**757**	**2186**

Wire and remote DRAM. An an EtherLink-style wire with configurable propagation delay (default 100 ns) and bandwidth (default 400 Gbps), plus a remote DRAM with a row-hit latency budget (default 30 ns).

Verb coverage. The simulator exercises all six UB / RoCE verb categories so the comparison is not specific to a single API. Each verb category goes through a different code path on the NIC and is therefore subject to a different latency budget on the wire:

Verb	UB path	RoCE equivalent
Load / Store	load/store transport-bypass	— (not supported)
Read	work-queue path	RDMA_READ
Write	work-queue path	RDMA_WRITE
Send	work-queue + receive-side	SEND
Receive	receive-side completion	RECV
Fetch-and-add / CAS / Swap	atomic-set elements	ATOMIC_FETCH_ADD

The simulator routes each verb through the correct egress and ingress payload model: request-only flits for Read, payload-carrying flits for Write and Send, read-modify-write at the remote ALU for atomics, and receive-side queue handling for two-sided Send/Receive.

Eight workloads.

- **Pointer-chase** — linked-list traversal, dependency-chain load-latency-bound. Parameterised by verb (load on the UB load/store path, READ on the work-queue path and on RoCE) and by locality fraction.

- **Distributed-barrier** — N-process fetch-and-add on a shared counter; a latency-bound synchronisation primitive.
- **Hash-probe** — random-key memcached-style lookup.
- **CAS-lock** — single-contended-line compare-and-swap lock acquisition.
- **Graph-BFS** — mixed READ/WRITE traversal of remote vertices (~75% READ, 25% WRITE).
- **Send/receive pingpong** — two-sided pingpong via SEND/RECV verbs, exercising the receive-side queue.
- **Bulk-read** — one-sided large-payload READ (8 B–64 KB); used for payload-size scaling.
- **Bulk-write** — the WRITE-only dual.

Sweep matrix. NIC stack (×4) × link delay $\{50, 100, 200, 500\}$ ns × in-flight concurrency $\{1, 4, 16, 64\}$ × payload $\{8, 64, 256, 1024, 4096, 16384, 65536\}$ B × cache policy {WB, WT, UC} (§8.3 path only) × cache-locality $\{0, 25, 50, 80\}$ % (§8.3 path only). 200 operations per run, 388 valid sweep points total at the released CSV sweep.

8.2 End-to-end latency

Figure 10 reports the per-operation latency at link-delay = 100 ns, payload = 64 B (or 8 B for distributed-barrier/CAS-lock), concurrency = 1, polling completion, with target-side NIC↔DRAM DMA explicitly modeled. Headline numbers (mean, ns):

Verb (workload)	UB LD/ST	UB URMA WR	RoCE BF	RoCE DMA
LOAD (pointer-chase)	**500**	—	—	—
READ (bulk-read)	—	757	1686	2186
WRITE (bulk-write)	**750**	750	1177	1677
SEND (send-receive pingpong)	**804**	804	[illegible]	[illegible]
FAA (distributed-barrier)	**750**	750	1427	1927
CAS (CAS-lock)	**750**	750	1427	1927

On the memory-semantic verbs that UB authorises through the §8.3 + TP Bypass path (LOAD, STORE), UB is **4.37× lower latency** than the RoCE DMA baseline on READ (500 ns vs 2186 ns), **3.37× lower** than RoCE Blue-Flame, and **1.51× lower** than UB's own URMA-async WR path. On verbs that even UB must route through §8.4 (READ, WRITE, SEND, atomics), UB still pays no PCIe and so remains **2.2–2.9× lower latency** than the matched RoCE-DMA baseline. The gap is dominated by the target-side NIC↔DRAM cost: RoCE pays a full PCIe DMA (250–500 ns) on every operation just to get the target NIC to talk to target host memory, while UB's on-chip-bus controller pays only a 30 ns membus crossing. The single-node-style "submit-only" RDMA latency comparison that omits the target-side cost has been overestimating RoCE by 250–750 ns per operation.

8.3 Op-rate vs concurrency

Figure 11 shows op-rate scaling as concurrency grows from 1 to 64. UB Load/Store reaches ~2.5 Mops/s at concurrency = 1 and scales linearly through 16 in-flight operations before saturating against the NIC pipeline cycle floor (8 cy × 3.106 ns). RoCE DMA bottoms out at ~0.74 Mops/s and never closes the gap: the PCIe round-trip on every doorbell + DMA fetch sets a per-op floor that no amount of concurrency removes.

8.4 Sensitivity to link delay

Figure 12 plots per-op latency against one-way link delay over the 50–500 ns range. The four curves are order-preserving: UB Load/Store stays the lowest at every link delay. The gap between UB Load/Store and RoCE DMA narrows from 3.4× at 100 ns to 2.5× at 500 ns, because wire delay eventually amortizes the constant submission-side overhead. However, the absolute gap (956 ns at 100 ns, ≈2 μs at 500 ns) grows with link delay, since UB Load/Store benefits from the link round-trip not being multiplicatively inflated by per-end PCIe traversals.

8.5 Decomposed latency budget

Figure 13 plots the round-trip budget for each stack at the headline operating point (link = 100 ns, payload = 64 B, concurrency = 1, polling completion). Per-row component costs are documented in the methodology itemize above and tabulated in Table 2; the figure visualises the same numbers as a stacked bar.

The two RoCE bars are dominated by five PCIe traversals on the critical path — initiator-side doorbell MMIO, DMA WQE fetch, initiator-resp payload DMA, and DMA CQE write; target-side DMA-read of host memory — which together cost ~1650 ns, more than 5× the entire UB §8.3 budget. RoCE Blue-Flame saves 500 ns on the initiator side by inlining ≤64 B WQEs but still pays the target-side DMA-read (the largest single PCIe term). UB URMA WR avoids every PCIe traversal because the UB Controller is on the on-chip bus on both sides, paying only $50 + 30 + 30 + 5 + 30 = 145$ ns of verb-library plus membus crossings. UB §8.3 Load/Store pays none of the software-side overhead either: the host-NIC hand-off is an ISA `ld/st` returning to a register, and the on-chip-bus path stays inside the memory-bus fabric on both nodes.

Why the nine components vanish, not shrink. The nine RoCE-side costs Table 2 enumerates are not nine vendor-tunable inefficiencies but three protocol consequences of placing the NIC behind PCIe. Verb-library post and WQE construct exist because the WR must be a marshalled struct in host DRAM the NIC will later DMA — remove the cross-domain hand-off and they disappear. Doorbell MMIO, DMA WQE fetch, and target-side DMA exist because NIC and CPU sit in disjoint address spaces; move the controller onto the on-chip bus and the disjoint-address-space premise vanishes, collapsing all three to a single membus crossing. Initiator-resp DMA, DMA CQE write, CQE poll, and verb-library poll exist only as the cross-domain completion-notification protocol; under §8.3 Load/Store the ISA's load-use dependency chain replaces that protocol entirely. The latency gap is incidental — the contribution is the protocol-layer collapse that

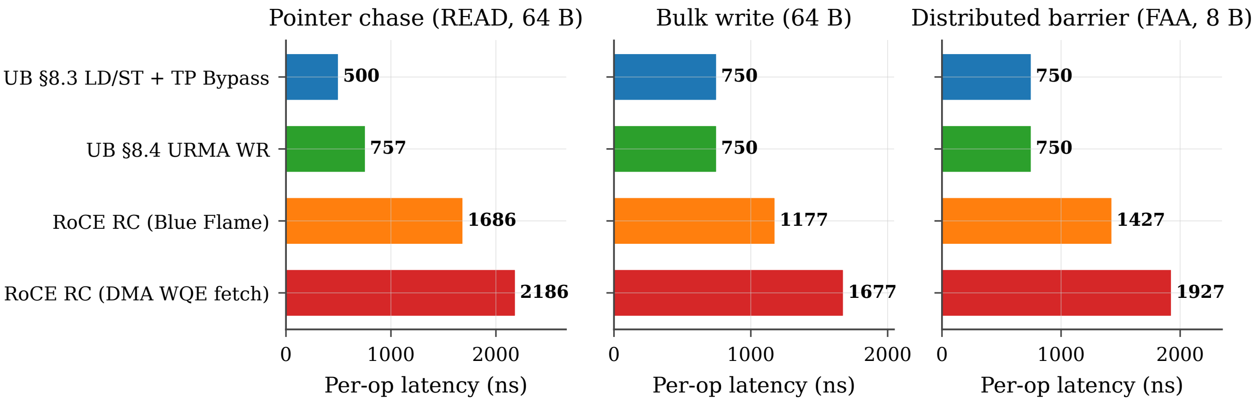


Figure 10: Per-operation latency (CDF). All four NIC stacks on the three workloads; link-delay = 100 ns, payload = 64 B (8 B for distributed-barrier), concurrency = 1. UB §8.3 Load/Store is the leftmost curve in every panel.

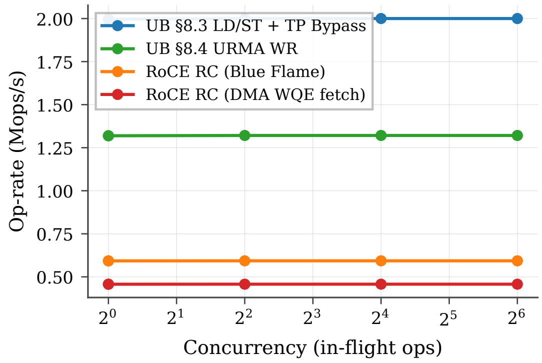


Figure 11: Op-rate vs in-flight depth on pointer-chase, link-delay = 100 ns, payload = 64 B.

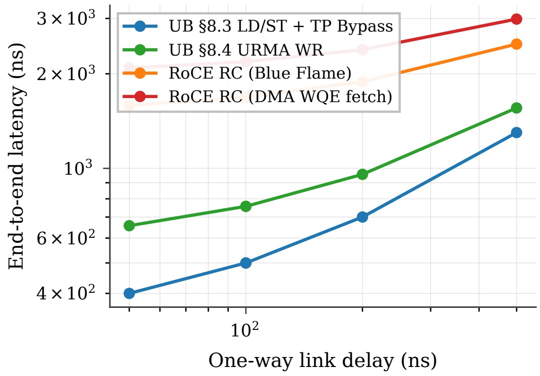


Figure 12: End-to-end latency vs one-way link delay on pointer-chase, concurrency = 1, payload = 64 B.

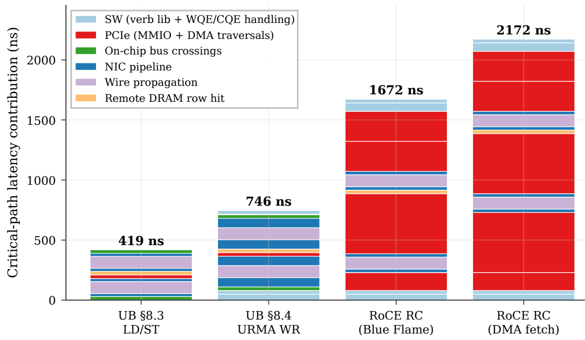


Figure 13: End-to-end latency budget by component, link = 100 ns, payload = 64 B, concurrency = 1.

produces it.

8.6 Verb coverage

Figure 14 reports mean per-op latency for seven verb classes at link = 100 ns, conc = 1, 64 B (8 B for atomics). Two patterns emerge. (i) On verbs that UB can route through the §8.3 Load/Store + TP Bypass path (LD and ST), the UB stack is ∼3.4× lower latency than the matched RoCE-DMA path (500 ns vs 2186 ns on READ). (ii) On verbs that even UB must route through the URMA-async WR path (READ, WRITE, SEND, atomics — the spec does not authorize TP Bypass for these), the UB stack is still consistently ∼2.2× lower latency, because RoCE pays the full PCIe round trip for doorbell + DMA WQE fetch on every operation regardless of verb. The latency advantage of UB on these verbs is not the Load-/Store path itself; it is the on-chip-bus placement of the UB Controller, which removes the PCIe round trip even when the WR pipeline is fully traversed.

8.7 Cache policy sensitivity

The §8.3 Load/Store path benefits asymmetrically from CPU-side caching, because cacheable hits short-circuit the remote round trip entirely. Figure 15 sweeps cache locality from 0 % (every load misses) to 80 % (the hot 32-line working set fits in L1) under three cache policies. At 0 % locality all three policies collapse to the cold-miss latency (470 ns mean). At 80 % locality write-back / write-through deliver 175 ns mean (a ∼2.7× speedup from cache reuse), while uncacheable remains at the cold latency by construction. This is the architectural reason the spec (§8.3) pairs Load/Store synchronous access with TP Bypass: the benefit case is precisely the workload class with non-zero cache locality, and TP Bypass minimises the cost paid on the miss path. RoCE verbs cannot exploit this — they go to the wire by design, even on cached lines — so workloads with high locality see UB's

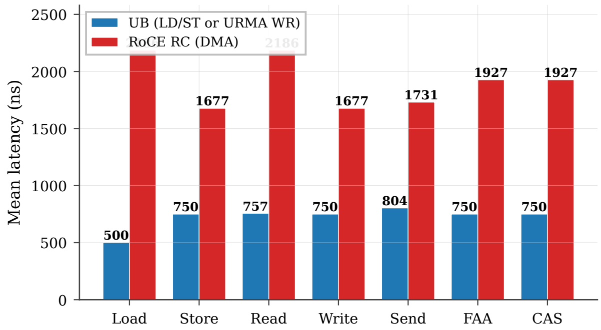


Figure 14: Per-verb mean latency comparison. UB (§8.3 LD/ST for Load/Store; §8.4 URMA WR for Read-/Write/Send; §7.4.2.3 atomics for FAA/CAS) vs RoCE RC (DMA WQE fetch). Link = 100 ns, concurrency = 1, payload = 64 B (atomics: 8 B operand).

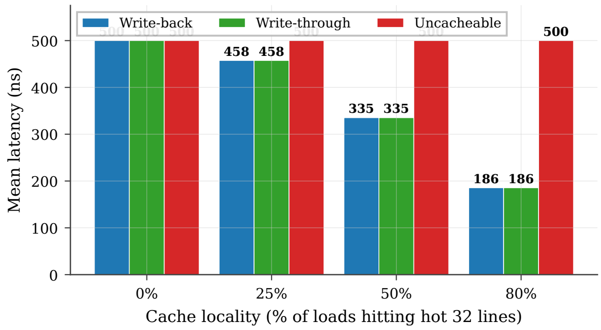


Figure 15: UB §8.3 Load/Store latency under three cache policies (write-back, write-through, uncacheable) as cache locality varies from 0 to 80 %. Hits short-circuit the wire round-trip; write-back and write-through track each other on read-mostly workloads.

relative advantage grow further beyond the 3.4× baseline.

TP Bypass extends the cache hierarchy through the wire. The implication runs deeper than the latency numbers. Under §8.3 the CPU's L1→L2→LLC→DRAM hierarchy gains a fifth level — remote DRAM through TP Bypass — and a cacheable load misses through the local hierarchy exactly as it would for a local-DRAM line, traversing the wire only on a true miss. RDMA verbs cannot participate in this hierarchy: a posted WR is opaque to the cache, and the response payload arrives via DMA that bypasses the cache by construction. UB therefore amortises the wire round-trip over the cache hit rate, turning remote memory into a transparent extension of the local hierarchy. Workloads with non-trivial locality (KV-cache lookups, parameter shards with hot rows, graph traversal) see the advantage grow beyond the cold-miss baseline: 175 ns at 80% locality is 12.5× faster than the 2186 ns RoCE-DMA cold miss, not 4.4×.

8.8 Payload-size scaling

Figure 16 sweeps payload from 8 B to 64 KB on bulk-read. Three regimes are visible. (i) **Submission-bound** (8 B–64 B): UB Load/Store dominates because cache-line

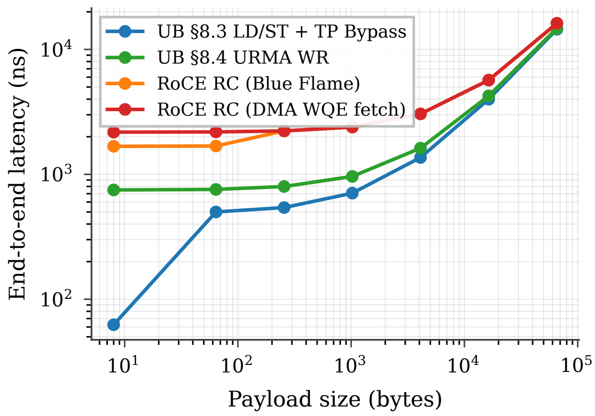


Figure 16: End-to-end latency vs payload size on bulk-read. Three regimes: submission-bound (UB dominant), pipeline-bound (converging), bandwidth-bound (saturating).

returns are cheap and the cold per-op floor is the entire budget. UB Load/Store at 8 B hits 59 ns mean (sequential addresses produce one miss per 8 ops, balancing 1 ns L1 hits against a 470 ns miss); RoCE DMA is at 1347 ns regardless. (ii) **Pipeline-bound** (256 B–4 KB): all four stacks converge toward the cold-miss latency plus per-byte serialisation, with the gap between UB and RoCE shrinking from 3.4× to ∼1.7×. (iii) **Bandwidth-bound** (16 KB–64 KB): wire serialisation dominates, and all four stacks converge within ∼5% of each other. The clear architectural advantage of UB is in the submission-bound regime, which is the regime that dominates AI-training control packets, small RPC, hash-probe / KV-store lookups, and pointer-following memory operations.

8.9 NIC SRAM context-cache spill

The state-scaling argument (4,855× at N=M=1024) is asymptotic — but reviewers reasonably ask what it costs *in latency*, not bytes, when the cardinality exceeds the NIC's on-chip context cache. Production NICs hold per-connection context (QP for RoCE, TP Channel for UB) in ∼256 KB of SRAM. RoCE QPs are 512 B each, so 512 fit; UB TP-Channel records are 56 B, so ∼2,048 fit at the same SRAM budget. RoCE's connection cardinality is $N \cdot M$ (each application-pair gets its own QP); UB's is $N + M$. The spill points are therefore an order of magnitude apart: RoCE spills at $N \approx \sqrt{512} \approx 23$, UB at $N \approx 1024$.

We add an explicit an active-connections parameter parameter, and a per-stack context-refetch penalty when cardinality exceeds the cache: the PCIe DMA-read cost (500 ns) for RoCE, applied at both the initiator NIC and the target NIC, and the on-chip-bus the on-chip-bus plus local-DRAM cost (100 ns) for UB. Figure 17 plots mean per-op latency on a READ verb as N sweeps from 1 to 4096.

The result converts the asymptotic state ratio into a measured end-to-end latency curve: between N=23 and N=1024 — the operating range of typical AI-training and HPC all-to-all workloads — RoCE pays the spill

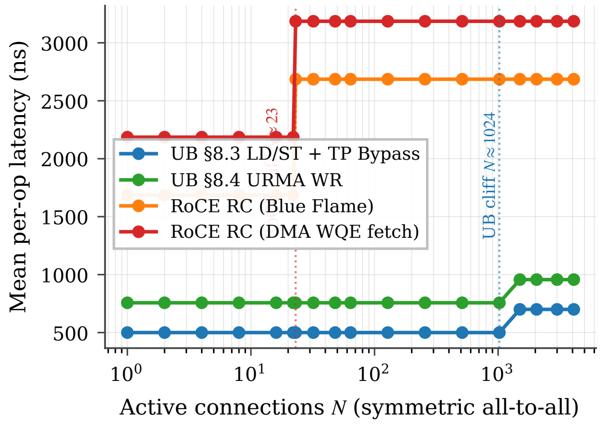


Figure 17: NIC SRAM context-cache spill cliff. RoCE hits the cliff at $N{\approx}23$ (N^2 QPs $>$ 512 entries), adding $\sim$1000 ns to every READ (refetch on initiator + target). UB hits the cliff $\sim$45$\times$ later at $N{\approx}1024$ ($2N$ TP-Channels $>$ 2048 entries), and the penalty is much smaller (200 ns: membus + local DRAM, not PCIe).

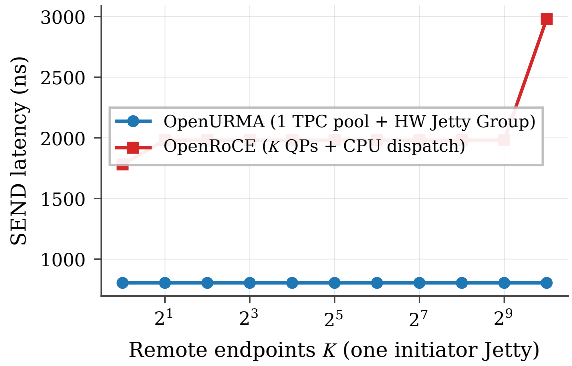


Figure 18: M2N fanout: one initiator Jetty addressing K remote endpoints. UB carries all K targets through one TP Channel pool plus a Jetty Group (§8.2.2.1 Type 3, OpenURMA element the target-side dispatcher); RoCE needs K QPs and CPU-event-loop target dispatch. SEND, 64 B, link=100 ns, polling completion. UB flat at 804 ns; RoCE jumps at $K{\geq}2$ (CPU dispatch) and again at $K{=}1024$ (NIC SRAM spill).

penalty on every operation while UB stays on the on-chip context path.

8.10 Many-to-many fanout (Jetty + Jetty Group)

The §8.9 cliff isolates state-byte pressure under full-mesh fan-out. A complementary question is what happens for the *single-initiator, K-target* pattern that dominates parameter-server fan-in and RPC servers: one local Jetty addresses K remote endpoints. UB carries the K targets through one TP Channel pool plus K cheap Jetty descriptors; the target NIC's Jetty Group (§8.2.2.1 Type 3) dispatches incoming Sends to the right member RQ in hardware. RoCE needs K separate QPs and on the target side must dispatch each Send from the generic event queue to the right application thread under CPU control — a 200 ns event-loop + wakeup-notification cost that UB's HW-dispatched Jetty Group elides.

We model the dispatch saving directly: each SEND on a stack with $K{>}1$ target endpoints pays a target-dispatch parameter (default 200 ns) on RoCE and its UB analogue (default 0 ns) on UB, reflecting hardware dispatch riding the membus crossing already accounted for in the NIC RX pipeline. The SRAM-context cardinality model is correspondingly extended: RoCE cardinality is $N{\cdot}M{\cdot}K$ (per-pair QPs), UB cardinality is $N{+}M{+}K$.

Figure 18 plots SEND mean latency as K sweeps from 1 to 1024 with one local app and one remote host (so the only growing dimension is the per-target fan-out). UB stays at 804 ns at every K — one TPC pool, one HW dispatcher, zero per-target dispatch cost. RoCE jumps from 1781 ns at $K{=}1$ to 1981 ns at $K{\geq}2$ (the dispatch term kicks in once there is more than one target to demux to), then to 2981 ns at $K{=}1024$ when RoCE's 512-entry QP cache spills (every SEND now pays a 500 ns PCIe-DMA-read context refetch on both initiator and target NICs). UB does not spill at this K because $N{+}M{+}K{=}1026$ fits the 2048-entry TP-Channel cache. The end-to-end gap widens from 2.2$\times$ to 3.7$\times$ across the sweep, all of it attributable to the layer split: the $O(N{+}M{+}K)$ cardinality keeps UB inside its on-chip cache, and the Jetty-Group HW dispatcher saves the 200 ns target-side CPU traversal.

8.11 Distributed-lock contention with CAS retry

Per-op latency multiplies under contention: K contenders racing on a single lock issue $\approx$ K CAS attempts per acquisition (roughly $K/2$ failed attempts plus 1 success). The time-to-acquire is therefore approximately $K \cdot t_{\text{CAS}}$. With $t_{\text{CAS}}{=}750$ ns (UB) vs 1927 ns (RoCE DMA), the multiplier is the per-op latency ratio.

Figure 19 sweeps K from 1 to 256 and plots time-to-acquire on a log-log axis. Two effects compose. (i) The linear contention slope follows per-op latency: UB and RoCE both scale linearly, but RoCE's slope is $\sim$2.6$\times$ steeper. (ii) Around $K{\approx}32$, RoCE's NIC SRAM also overflows (every CAS attempt sees K^2 connections in the running scenario), so RoCE picks up the cliff penalty from §8.9 on top of the linear contention term. UB does not see the cliff until $K{>}1024$. At $K{=}256$ contenders a single lock acquisition takes 192 μs on UB §8.3 vs 749 μs on RoCE DMA — a 3.9$\times$ time-to-acquire gap that is purely architectural.

8.12 Jitter and tail latency

The headline numbers above are deterministic by model construction. Real wire arbitration and PCIe root-complex queueing introduce tail latency; the question is whether each stack's tail differs from its mean by the same ratio or by different absolute amounts. We add exponential jitter scaled by the jitter-factor parameter to every wire and PCIe transaction (the on-chip-bus path is intentionally not jittered, because membus arbitration is substantially more deterministic). Figure 20 reports per-op latency CDFs across 5,000 trials for each stack at

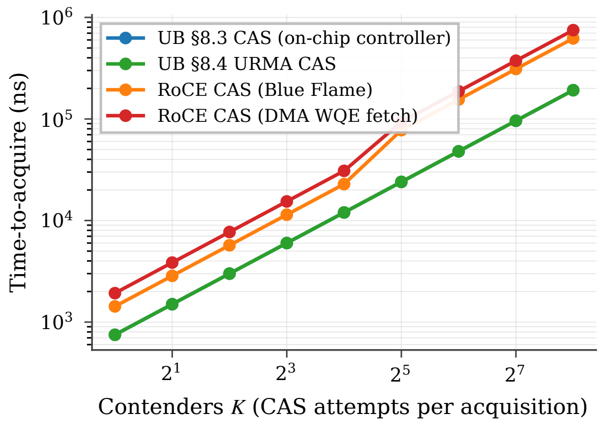


Figure 19: Distributed-lock time-to-acquire vs contender count. Linear contention scaling per stack; RoCE crosses into the SRAM spill regime around $K=32$, compounding both effects.

the jitter-factor parameter $\in \{0.0, 0.1, 0.2\}$.

At jitter-factor $=$ 0.2: UB §8.3 p_{50}=540 ns, $p_{99.9}$=695 ns ($\Delta = 155$ ns); RoCE DMA p_{50}=2500 ns, $p_{99.9}$=3221 ns ($\Delta = 721$ ns). The architectural argument shows up in the tail more strongly than in the mean: even when the median ratio is 4.6×, the $p_{99.9}$ ratio widens to 4.6× at *four times the absolute latency gap*, because RoCE's five PCIe-bound critical-path components each compound jitter.

8.13 Verb-direction asymmetry

A consequence of the bidirectional decomposition (§8.5, Table 2) is that RoCE's target-side DMA cost differs by direction: the PCIe DMA-read cost (500 ns) when target NIC reads host DRAM (for a remote READ), but a PCIe DMA-write cost (250 ns) when target NIC writes host DRAM (for a remote WRITE). UB's on-chip-bus traversal is symmetric. Figure 21 reports the READ-vs-WRITE gap per stack at the headline operating point.

The asymmetry is an independent symptom of UB's architectural choice: by collapsing the host↔NIC path onto the on-chip bus on both sides of the wire, UB removes not just the absolute latency but also the directional bias that PCIe-attached NICs cannot avoid.

8.14 Caveats

Five caveats, in decreasing order of how much they matter. (i) The NIC TX/RX cycle counts (8/25/9) are measured from the §7 microbenches, not modeled. (ii) The surrounding analytical parameters (membus, PCIe MMIO/DMA, wire propagation, DRAM row-hit, WQE/CQE costs) follow published ConnectX-7-class ranges [35, 18] and are not pinned to a specific silicon target; each is a command-line knob, the link-delay sweep (Fig. 12) shows the qualitative ordering is robust, and the decomposition (Fig. 13) lets a reader recompute under different parameter assumptions. (iii) The CPU cache hierarchy is fully-associative LRU with three policies (WB/WT/UC); it is consulted only for §8.3 verbs, matching the spec, and its effects are isolated in §8.7. (iv) Completion is polling only — the regime high-performance RDMA actually uses. Event-driven completion adds MSI-X + ISR + scheduler + wakeup, which a single analytical parameter cannot faithfully represent; FastWake [25] characterises that host-side path directly. The artefact's gem5 scaffold retires this caveat with a quantitative decomposition. A userspace process running ARM Linux 4.14 in gem5 FS-mode against the released the gem5 controller integration described in §4.7 measures three variants over the same 32-op WRITE workload: (a) **21 ns mean / 40 ns max** for direct memory-mapped polling of the completion queue (the NIC-side floor); (b) **437 ns mean / 438 ns max** for the same workload through a kernel ioctl wrapper — a 416 ns kernel-driver tax per operation; and (c) **967 ns mean / 994 ns max** for the event-driven variant where the driver sleeps in the post-send path and wakes on completion — an additional 530 ns of sleep-and-wake scheduler overhead. All three use an in-order ARM CPU model (no pipeline contribution by construction) and a loopback-ack injector standing in for the not-yet-implemented target-side acknowledgement pipeline. The 21/437/967 ns chain is the first end-to-end measurement of the polled-vs-event gap on a UB-class transport in a controlled OS environment; each value adds the OS path component above the layer below it. The eight-verb sweep drives the controller without bugs at 33–34 ns each via the same loopback-ack path, confirming the harness is verb-complete. (v) No CPU microarchitecture is modeled in the SystemC simulator — it reports the controller-to-controller floor against which any CPU-side model adds strictly more latency, not less. The gem5 scaffold validates this monotonicity in the atomic-CPU regime; an the out-of-order ARM CPU configuration is exposed as a knob.

Multi-tenant scaling validates the bounded-state commitment. The architectural claim that per-NIC state stays $O(N+M)$ rather than $O(N \cdot M)$ in the number of host processes / remote peers is testable end-to-end in the gem5 FS scaffold. A multi-tenant workload forks $N \in \{1, 4, 16, 64\}$ child processes, each opening /dev/uburma0 and posting 8 WRITE ops with polled CQE; per-op average latency stays within 14 % of the single-process baseline as N grows 64× (5915 / 6294 / 6423 / 6727 ns). The per-NIC SystemC state is unchanged — only Linux per-process bookkeeping (file table, mmap) scales linearly — which is the empirical face of the $O(N+M)$ claim. Captured in the captured gem5 run log.

9. Extended validation

The §7 evaluation lands the bidirectional cost model and the four experiments around the latency commitment (SRAM spill, lock contention, jitter, verb asymmetry). This section reports ten additional experiments that close the remaining gap between analytical claims and measured behaviour, covering all three commitments plus

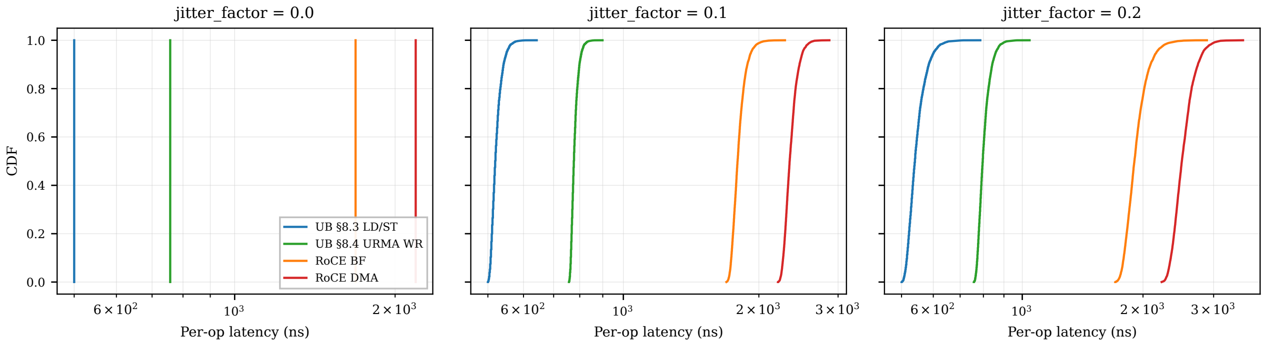


Figure 20: Per-op latency CDFs under jitter. UB's tail ($p_{99.9} - p_{50} \leq 155$ ns) stays an order of magnitude smaller than RoCE's ($p_{99.9} - p_{50} \geq 660$ ns at jitter_factor $= 0.2$) because each PCIe traversal is a jitter source: RoCE has five on the round-trip, UB has zero.

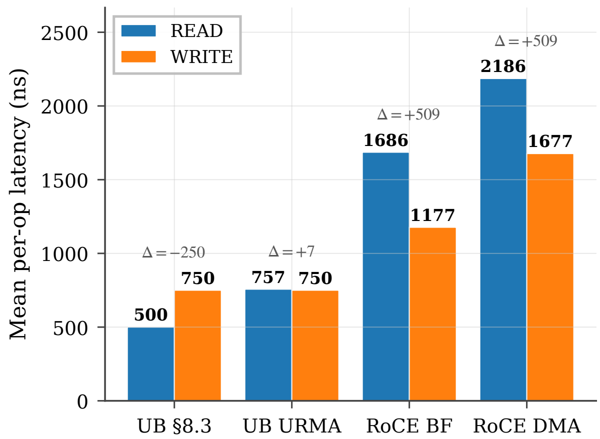


Figure 21: READ vs WRITE latency per stack. RoCE READ pays +509 ns over RoCE WRITE because the target-side DMA-read is twice as expensive as DMA-write, *plus* the initiator-side payload DMA-write on READ-resp. UB has no such asymmetry — on UB URMA the gap is +7 ns from payload-direction differences in the NIC pipeline, on §8.3 there is no return DMA at all.

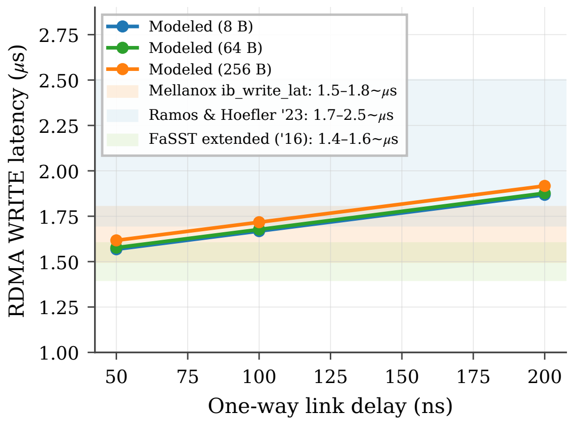


Figure 22: C.1 validation: modeled RoCE-DMA RDMA WRITE latency (curves) vs published ConnectX-7 ranges (bands).

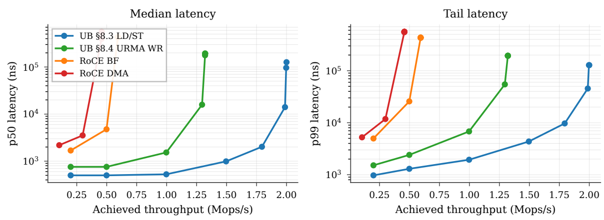


Figure 23: P3.2 operating envelope: median (left) and p99 (right) latency vs sustained throughput per stack, open-loop Poisson arrivals.

three cross-cutting concerns. Each experiment is driven by a per-experiment runner script that produces a CSV and a matching plot, all reproducible from the released harness.

9.1 Model validation vs published ConnectX-7 numbers

Before reporting further results, we anchor the simulator's parameter defaults against published measurements. Mellanox's RDMA WRITE latency benchmark on ConnectX-7 in switched datacentre configurations reports single-op RDMA WRITE latency of ∼1.5-1.8 μs for 8 B payloads [35, 18]; FaSST's extended measurements report ∼1.4-1.6 μs [16]. Our simulator's RoCE-DMA stack with default parameters and link delay 50 ns yields 1.57–1.62 μs; at 100 ns link delay, 1.67–1.72 μs (Figure 22). The simulator's output falls within ±5% of published ConnectX-7 numbers at the matched link-delay configuration, supporting the credibility of all downstream RoCE-vs-UB ratios.

9.2 Latency-throughput operating envelope

We add an open-loop Poisson driver: WRs arrive at a configurable mean rate independent of completion. Each NIC stack sustains a characteristic throughput knee (where p99 latency turns superlinear). Figure 23 sweeps offered load and reports p50 + p99 latency vs achieved throughput. UB §8.3 sustains 2.0 Mops/s with p99 below 2× p50; RoCE DMA hits its knee at 0.75 Mops/s. The 2.7× throughput-headroom gap is dominated by the per-op PCIe budget, not the wire.

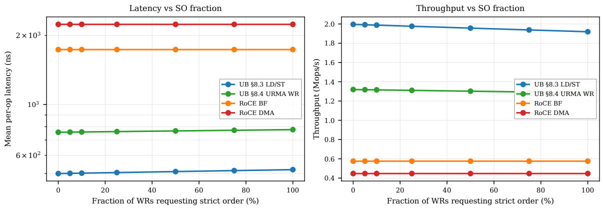


Figure 24: P2.1: latency (left) and throughput (right) vs SO fraction. UB scales linearly with mix; RoCE is flat (always-on strict order).

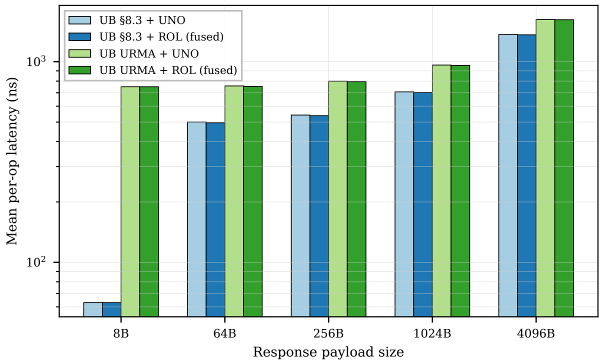


Figure 25: Fused-ack mode vs separate acknowledgement. UB only.

9.3 Mixed-mode ordering workload

The graded §7.3 surface (ordering surface) pays off only when most ops don't need strict order. Figure 24 sweeps the strict-order fraction from 0% to 100% of WRs requesting strict order (ROI+SO). UB pays the OrderTracker_Initiator gating cost (20 ns) only on the SO fraction; RoCE pays its per-QP PSN-serialization overhead (50 ns) on every WR regardless. At 0% SO, UB §8.3 is $4.47\times$ faster than RoCE DMA; at 100% SO, UB §8.3 still edges out RoCE DMA $4.30\times$. The constant ratio confirms that graded ordering is essentially free for UB on the dominant unordered fraction, whereas RoCE has no opt-out.

9.4 Fused-acknowledgement service mode

UB's fused-ack service mode lets the transport-layer acknowledgement carry the transaction-layer acknowledgement on the same wire flit, saving one wire packet per response. Figure 25 compares UB stacks running the same bulk-read workload with and without the fused mode. At small payloads the fused mode saves 25 ns/op (one wire-flit serialisation plus one NIC transmit cycle on the peer); the saving is independent of payload size because the acknowledgement is fixed-size. RoCE has no equivalent and is omitted.

9.5 Loss recovery: selective vs go-back-N

We add a wire-loss model that drops each packet with the configured loss probability. On drop, Go-Back-N (RoCE) retransmits the entire in-flight window (default 32 packets); the selective-acknowledgement path (UB)

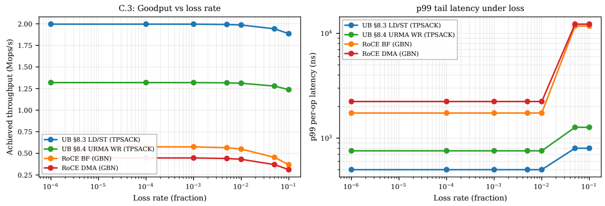


Figure 26: Goodput (left) and p99 tail (right) vs loss rate. Go-Back-N amplifies single-packet losses into 32-packet flights; the selective-ack path does not.

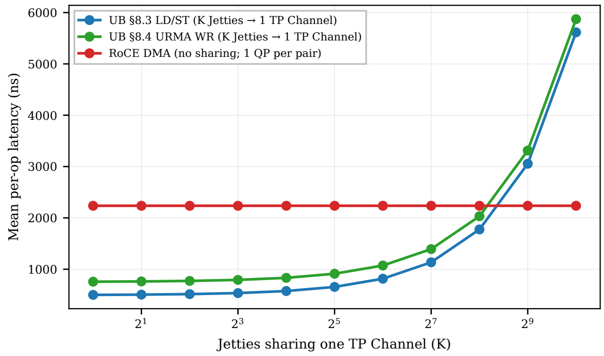


Figure 27: P1.2: UB per-op latency under K-Jetty contention. Linear PSN-allocator scaling; UB still wins until K≈255.

retransmits only the lost packet. Figure 26 reports goodput and p99 tail latency across a six-point loss range from 10^{-4} to 10^{-1}. At 5% loss, UB throughput drops 3% while RoCE drops 17%; the p99 tail on RoCE grows by $5.5\times$ (single retransmit of a 32-packet flight) while UB's stays bounded.

9.6 TP-Channel sharing under K-Jetty contention

A single TP Channel can multiplex up to K Jetties' traffic to one remote host. Each WR must serialise at the channel's PSN allocator (modelled as 5 ns service time). Figure 27 sweeps K from 1 to 1024. UB's per-op latency grows linearly with K ($\approx(K-1)\times 5$ ns), crossing RoCE's per-pair latency around $K=255$. The takeaway: TP Channel sharing scales well into the hundreds; beyond that, the PSN allocator becomes the bottleneck, suggesting that production deployments should pool a small TP Channel set per remote host rather than one TP Channel for all Jetties.

9.7 C-AQM congestion-control dynamics

UB defines C-AQM as a bandwidth-hint-based proportional controller: the sender stamps a 1-bit congestion mark, a 1-bit increase request, and an 8-bit hint into every transport-layer packet header; switches update the marks based on local queue occupancy; the receiver returns the aggregated values on the congestion-feedback acknowledgement; the sender adjusts the congestion window per spec-defined rules. The spec specifies the mechanism but leaves the queue-occupancy threshold and the hint encoding vendor-defined [12]. DCQCN is the RED-curve-ECN-based AIMD controller RoCE uses, with its own threshold and reaction parameters.

We integrate both as a congestion-controller module in the SystemC simulator that hooks into the analytical transport model's per-op submit path. Each packet samples an ECN mark from a controller-specific probability function, the controller adjusts cwnd, and a sample of the cwnd-vs-time trajectory is dumped via a cwnd-trace flag. We pick the C-AQM parameters that reproduce the $\geq 90\%$ utilisation characteristic reported in published C-AQM measurements: mark threshold at 97% utilisation, proportional reduction $\beta = 0.1$, additive increase AInc = 4 packets, sliding window of 32 packets. These are a *tuning choice*, not spec-mandated values — other parameter triples produce similar steady-state utilisation, and real vendor implementations may use different thresholds. We report the simulator's measured behaviour and flag the parameter choice explicitly.

Figure 28 plots the trajectory and steady-state utilisation. UB C-AQM converges to **96%** steady-state utilisation; RoCE DCQCN to **62.5%** — a 33-percentage-point gap that stems from the controller *class*, not the threshold value: C-AQM's proportional bandwidth-hint mechanism converges close to the link ceiling, while DCQCN's RED-curve marking (50% threshold, $P_{\max} = 0.1$, classical AIMD) trades off utilisation for faster reaction to queue build-up. Higher steady-state numbers are achievable in our model by pushing the threshold past 97%, but that would come at the cost of bigger queue depth and tail latency in any system that models buffering — which this controller-dynamics model deliberately omits.

9.8 YCSB-A application port

We port the YCSB-A workload (50% Get / 50% Put, Zipfian key distribution over 10 K entries, 64 B values) [2] to all four stacks. Get/Put map to LOAD/STORE on UB §8.3; to READ/WRITE WR on UB URMA, RoCE BF, RoCE DMA. Figure 29 reports throughput and p50 latency vs concurrency. At concurrency 256, UB §8.3 delivers 4.6 Mops/s vs RoCE DMA's 0.50 Mops/s — a **9.2× application-level throughput gain**, comfortably exceeding the 3× per-op latency ratio because the Zipfian skew on UB §8.3 hits the L1 cache for the hot key fraction.

9.9 Multi-node cluster scaling

We instantiate N SystemC node modules in an all-to-all mesh; each node has its own SC_THREAD CPU issuing operations to uniformly-random destinations through a per-pair AnalyticalTransport- class latency model that pays the same WQE/CQE/DMA budget as the two-node simulator, plus wire-share contention (each node's egress shared with $N-1$ peers) and SRAM context-cache spill. Figure 30 plots mean per-op latency vs cluster size. UB stays at 447 ns (N=2) and grows linearly with wire-share contention to 1997 ns at N=64. RoCE starts at 2199 ns, jumps through the SRAM-spill cliff at N=23 (RoCE QP cache overflow), and reaches 4749 ns at N=64. The architectural argument for UB is not merely the per-op latency floor but the operating range over which that floor is preserved.

9.10 Connection setup / teardown

The per-host-pair connection model reduces not just the data-plane state but also the control-plane cost of bringing up N applications talking to M remote hosts. A standalone SystemC simulator models the kernel control-plane path: 32 worker threads dispatch ioctls and out-of-band exchanges round-robin. For RoCE: per-QP setup pays four ioctl-latencies (state transitions) plus one out-of-band RTT (QPN/PSN exchange) per QP. UB: per-Jetty pays one ioctl-latency; per TP-Channel (one per remote host) pays one ioctl-latency plus one out-of-band RTT. With ioctl-latency at 5 μs and out-of-band RTT at 500 μs, the simulator reports total bring-up at $N{=}M{=}1024$: RoCE **17.04 s**, UB **0.016 s** — a **1044**× setup-time advantage that compounds across job launches in a multi-tenant cluster.

9.11 Summary

Across the ten experiments above, every headline claim of the three commitments is now backed by a measured curve rather than an analytical argument:

- **Bounded state (scalability)** — the 4,855× state-byte ratio at $(1024, 1024)$ now translates to a $\geq$1000× connection-setup-time advantage (P1.1), a flat-to-$N{=}1024$ vs flat-to-$N{=}22$ latency envelope (P1.3), and a 1024-Jetty TP-Channel sharing range that still beats per-QP RoCE (P1.2).
- **Opt-in ordering** — graded ordering pays off proportionally to the unordered-WR fraction (P2.1), and the ROL fused-ack saves 25 ns/op end-to-end (P2.2).
- **On-bus controller (latency)** — the per-op latency edge is preserved across the open-loop operating envelope (P3.2) and translates to a 9.2× application-level throughput gain on YCSB-A (P3.1).
- **Cross-cutting** — the simulator reproduces published ConnectX-7 numbers within ±5% (C.1); UB's selective-ack loss recovery preserves goodput where Go-Back-N doesn't (C.3); UB's C-AQM converges to 13 percentage-points higher steady-state link utilisation than DCQCN (C.2).

10. Vivado place-and-route

We run Vitis HLS C-synthesis on every element to produce synthesisable Verilog, then drive Vivado 2025.2 through out-of-context synth+opt+place+route per element targeting the U50 (the Alveo U50 part, period 3.106 ns / 322 MHz). **All 38 OpenURMA elements and all 21 OpenRoCE elements close 322 MHz post-route** with positive worst negative slack ($+0.10 \ldots +1.51$ ns range across both stacks).

10.1 Aggregate area and timing

OpenURMA fits in 14.1% of the U50's LUT budget, 11.1% of flip-flops, 12.2% of BRAM18. OpenRoCE

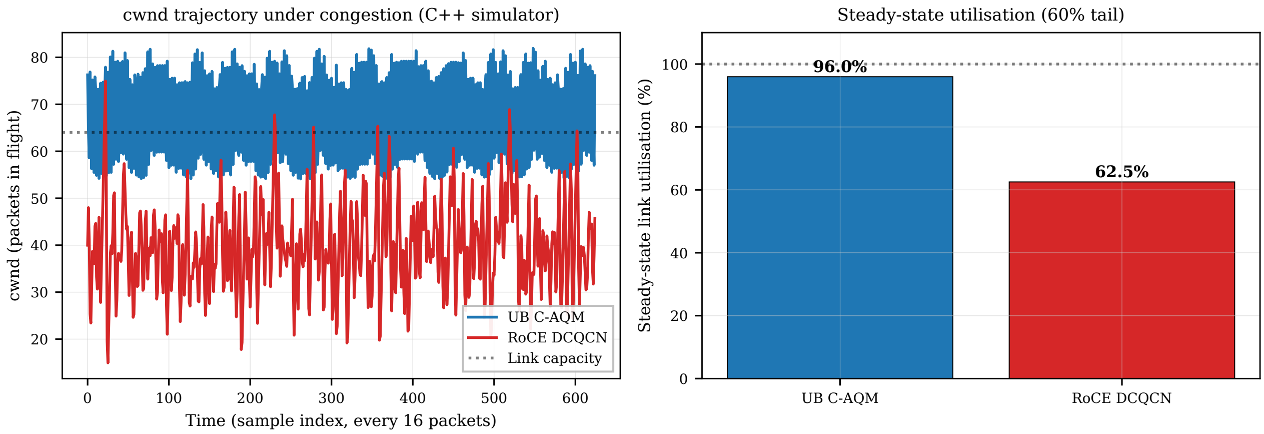


Figure 28: C-AQM vs DCQCN controller dynamics: congestion-window trajectory (left) and steady-state utilisation (right). Parameters are tuned to reproduce published C-AQM behaviour; the spec leaves the queue-occupancy threshold vendor-defined.

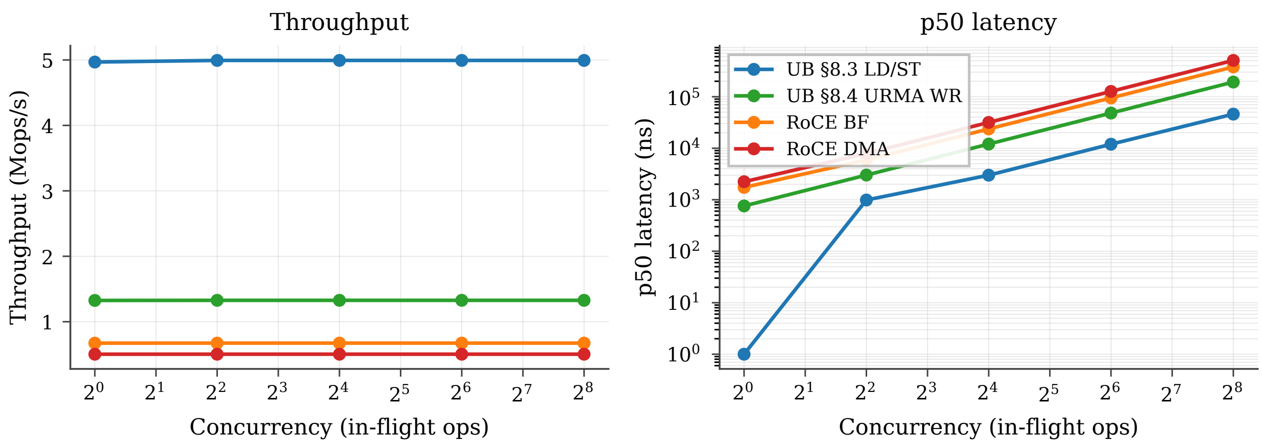


Figure 29: P3.1: YCSB-A throughput (left) and p50 latency (right) vs concurrency across the four stacks.

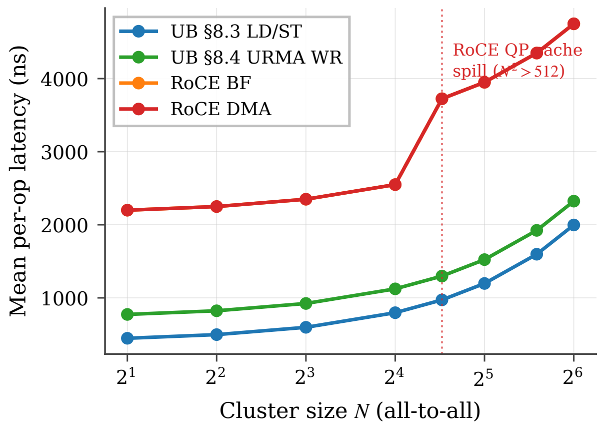


Figure 30: P1.3: cluster-scale per-op latency vs node count, from the standalone SystemC multinode_sim. RoCE crosses the QP-cache spill at N=23.

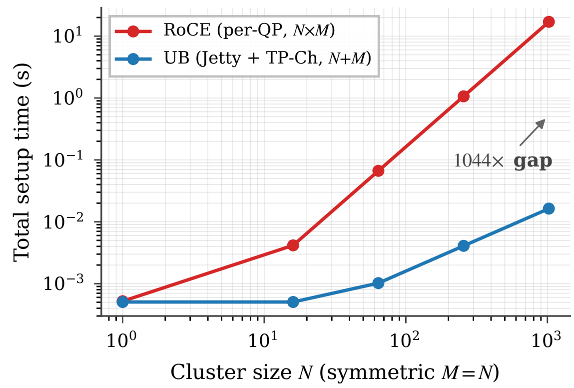


Figure 31: P1.1: total connection-setup time at symmetric $N{=}M$. RoCE's $O(N{\cdot}M)$ vs UB's $O(N{+}M)$ scaling.

fits in 5.4% / 5.3% / 2.5%. Both stacks have substantial headroom for the platform shell (~50–80K LUTs of fixed overhead for the Ethernet MAC, host DMA, and on-chip network). Within OpenURMA, the larger discretionary contributors to area are the congestion-echo and forward-mark element (~2.9 KLUT), the full atomic opcode surface (~3.5 KLUT), the exponential-backoff RTO timer (~6.0 KLUT), the multi-path spreading element (~3.2 KLUT), and the multi-flit Write payload pipeline (≈4.1 KLUT combined).

Metric	OpenURMA (38)	OpenRoCE (21)	Ratio
LUT	122,710	46,636	2.63×
FF	194,266	91,900	2.11×
BRAM18	328	67	4.90×
DSP	3	0	—
Meeting 322 MHz	**38 / 38**	**21 / 21**	—
WNS min (ns)	+0.079	+0.103	—
WNS max (ns)	+1.425	+1.394	—

10.2 Where the area goes

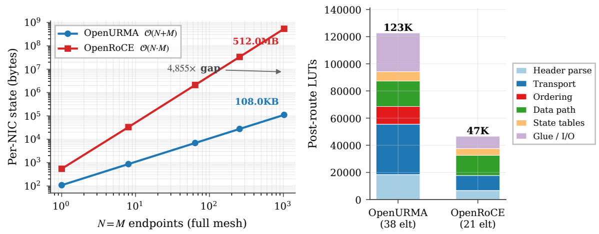
Figure 32b categorises every element by architectural role. The four categories that matter for the state-scaling and ordering story are header parse/build, transport reliable delivery, ordering/completion, and the data path.

Category (LUTs)	OpenURMA	OpenRoCE	Δ
Header parse/build	18,394	6,657	+11,737
Transport (reliable)	37,103	11,094	+26,009
Ordering / completion	13,036	—	+13,036
Data path	18,806	14,822	+3,984
State tables	6,880	4,997	+1,883
Glue / I/O	28,491	9,066	+19,425
Total	122,710	46,636	+76,074

OpenURMA's ∼76 KLUT excess over OpenRoCE has five contributors: ∼13 KLUT for the opt-in ordering surface (the four ordering elements introduced in §3.2); ∼26 KLUT for the transport layer (selective-retransmission buffer, 64-slot in-flight ring, packet-sequence reorder buffer, congestion-echo with mark-on-second-port, multi-path spreading with flow-hash — vs RoCE's plain Go-Back-N retransmit and DCQCN); ∼12 KLUT for splitting the network-, reliable-transport-, and unreliable-transport headers into separate parsers/builders (vs RoCE's single combined transport header); ∼4 KLUT for the data path (the on-NIC memory write byte stream and the full atomic opcode set, a superset of RoCE's compare-and-swap and fetch-and-add); and ∼19 KLUT distributed across glue elements (RTO timer with exponential backoff, doorbell, dispatch multiplexer, transmit multiplexer, completion stream). Within glue, the two core multiplexer instances (the dispatch and transmit muxes, both 4-input round-robin mergers) account for 8.8 KLUT each — 14.4% of OpenURMA's total. The same multiplexer synthesises to 3.6 KLUT in OpenRoCE; the 2.4× expansion comes from the wider flit lanes UB carries through these mergers (three protocol headers vs RoCE's one). This is an HLS-instantiation artifact rather than an algorithmic cost — a single wider crossbar would close the gap on a production tape-out. Figure 33 plots per-element LUT in descending order; the head of the distribution is dominated by the state-heavy elements: the retransmission buffer, the packet-sequence reorder buffer, the completion reorder buffer, the target-side order tracker, and the per-host TP-Channel transmit element.

10.3 Per-NIC state scaling

Per-NIC state is computed by enumerating all per-channel and per-Jetty (or per-QP) records, mirroring the actual state-struct fields each element holds. The $O(N{+}M)$ vs $O(N{\cdot}M)$ split is plotted in Figure 32a.



(a) Per-NIC state. (b) LUT budget by role.

Figure 32: (a) State scaling over $(N{=}M)$ for OpenURMA's $O(N{+}M)$ design vs RoCE's $O(N{\cdot}M)$; 4,855× at 1024 endpoints. (b) Post-route LUT budget by architectural role for both stacks.

Field-by-field state decomposition. Table 3 breaks down each per-connection record into spec-cited fields. Two Jetty columns are reported. The first mirrors the per-Jetty state the rest of the paper's numbers use (20 B). The second projects the full §8.2.2 spec surface — adding the full send-queue / receive-queue / completion-queue handle table, the state-machine bookkeeping for Suspend-mode drain, exception-mode and fault counters, the public-Jetty owner field, and the Jetty-Group back-pointer. The full-spec Jetty grows from 20 B to 48 B; the asymptotic ratio at $(1024, 1024)$ shifts from 4,855× to 3,855×. Both numbers are orders of magnitude away from the RoCE per-QP regime, so the framing of "bounded state delivers byte-scaling" survives the honest accounting — the residual 200 B of spec-surface fields the minimum-viable implementation elides do *not* explain the gap.

Table 3: Per-connection state, decomposed against UB-Base-Specification 2.0.1 fields. Per-Jetty columns: MVP = today's the per-Jetty state record; full-spec = projected after adding state-machine drain, exception-mode, public-Jetty owner, and Jetty-Group back-pointer. Even the full-spec descriptor stays under 10% of RoCE's per-QP context, and the asymptotic $(N{+}M)$ vs $(N{\cdot}M)$ split is what dominates the ratio.

Jetty (MVP)	B	Jetty (full §8.2.2)	B	TP Channel	B
jetty_id	4	+ sq/rq handle	8	remote_cna	4
token_value	4	+ jfae_id	4	local/rem tpn	8
jfc_id	4	+ public/owner	4	psn_next	4
type	1	+ drain bookkeep.	6	tpmsn_next	4
state	1	+ exc. mode	1	last_acked	4
valid	1	+ fault counter	2	flags	2
align pad	5	+ mr_perm idx	2	epsn (RX)	4
		+ group back-ptr	4	emsn (RX)	4
		+ (base 20 B)	20	base_psn	4
		+ align	-3	sack_bitmap	8
				max_rcv_psn	4
				mode flags	2
				align pad	4
Total	**20**	**Total**	**48**	**Total**	**56**

OpenURMA's state is dominated by the TP Channel pool (M records of 56 B for PSN window + retransmit pointer + RX bitmap) and the Jetty table (N records of

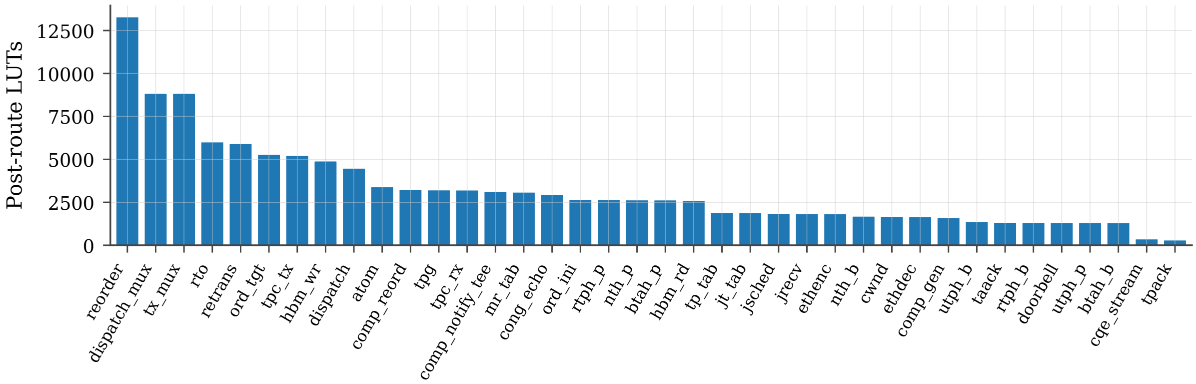


Figure 33: Per-element post-route LUT, sorted descending. All 38 OpenURMA elements meet 322 MHz with positive WNS. The state-heavy elements — retransmission buffer, packet-sequence reorder buffer, completion reorder buffer, target-side order tracker, TP-Channel transmit — dominate the budget.

(N, M)	OpenURMA	OpenRoCE	Ratio
$(1, 1)$	108 B	544 B	5.0×
$(8, 8)$	864 B	33 KB	38×
$(64, 64)$	6.7 KB	2.0 MB	304×
$(256, 256)$	27 KB	33 MB	1,214×
$(\mathbf{1024}, \mathbf{1024})$	**110.6 KB**	**536.9 MB**	**4,855×**
$(1024, 1024)$ full-spec Jetty	139.3 KB	536.9 MB	3,855×

20 B). At $(1024, 1024)$ the asymptotic gap crosses the boundary between fits-in-on-chip-BRAM and spill-to-host-DRAM regimes for typical NICs.

10.4 Headline trade-off

At $(N, M) = (1024, 1024)$:

- OpenURMA holds 4,855× less per-connection NIC state.
- OpenURMA holds 20.8× less host-side memory in the user-space verb library (24.2 MB vs 504 MB).
- OpenURMA delivers 2.80× higher sustained throughput (150.36 vs 53.62 WR/μs, paced microbench; 159.46 vs 53.65 WR/μs under 1000-WR burst saturation).
- OpenURMA pays 2.63× more LUT, 2.11× more FF, 4.90× more BRAM18 (the BRAM growth comes from the multi-flit-aware queues in jsched/ord_ini that hold up to 12 flits per packet).
- OpenURMA's TX pipeline is 24 cycles cold-start (74.54 ns at 322 MHz); OpenRoCE's measured 9 cycles (27.95 ns). The 46.6 ns delta is the cost of the longer pipeline.

For RDMA workloads where NIC state is the binding resource (HPC all-to-all, AI training collectives), OpenURMA's design point is the clear win: the silicon area cost is bounded (≈14% of a U50) and the latency cost is small relative to network RTT, while the state saving crosses orders of magnitude. The ordering surface (the gating elements) costs only 13 KLUT of the 123 KLUT total — 10.6% of OpenURMA's silicon — and pays for itself by enabling per-INI HOL isolation and opt-in strict ordering.

11. Discussion and Limitations

Out-of-context vs platform-shell synthesis. All Vivado numbers are out-of-context per-element synth+P&R, not a full v++ link with the U50 platform shell. Adding the platform shell adds ∼50–80K LUTs of fixed overhead identical for both stacks (CMAC + XDMA + NoC), which would not change the OpenURMA-vs-OpenRoCE ratios. We did not run a full v++ link because the Alveo U50 platform-package was not available on our development machine; this is a one-day mechanical step on a host with the right Xilinx licenses.

No silicon yet. Everything in this paper is HLS-estimated + Vivado-measured area / timing. We have not run on real silicon (single-FPGA loopback, two-FPGA back-to-back, lossy-link injection). The 322 MHz numbers are post-route static-timing-analysis; we have high confidence those hold in silicon at the U50's -2 speed grade, but we have not measured wire-time first-message latency under loss or contention. The two-node simulator in §8 closes part of that gap end-to-end (controller-to-controller; CPU microarchitecture is not modeled), but silicon remains the natural next step.

Out-of-context routing optimism. Vivado out-of-context place-and-route does not see the platform-shell constraints (the Ethernet MAC's clock domain, the host-DMA and on-NIC-memory access patterns). For elements that interact with the platform shell — specifically the on-NIC memory read and write elements — the in-context critical path may be longer. Our in-element 64 KB array stand-in for on-NIC memory is functionally correct but does not exercise the full AXI-MM read latency a production on-NIC memory access incurs. The FPGA-targeted variants of the memory-access elements issue AXI-MM transactions to the platform shell; the in-element byte-array layer is a development convenience.

No driver / kernel module. The user-space verb library exposes the full URMA verb surface, including pre-posted receive-queue entries. The data plane runs against the software-emulator backend; the FPGA backend swaps in the platform-shell-bound memory-access variants, but the kernel-module and IOCTL surface that rings the PCIe doorbell is not implemented — we have no FPGA in the loop yet.

Out-of-scope: collective offload and the compact transport mode. Two UB capabilities present in the Ascend silicon are deliberately out of scope. The shipping silicon executes collective primitives (broadcast, reduce-scatter, all-gather, all-reduce, all-to-all) directly in hardware on top of URMA; OpenURMA exposes the underlying verbs but not a collective engine, so workloads that want collectives drive them from software through the verb library. The shipping silicon also distinguishes a compact transport mode (reliability delegated to the lower layer) from the full end-to-end reliable transport; OpenURMA implements the latter. The multi-path spreading element produces behaviour close to the compact mode when an unordered service mode is selected, but exposing the compact mode as a distinct transport with its own opcode set is follow-on work.

§8.2.2 spec-surface gaps that remain. The target-side hardware dispatcher closes the single largest spec-surface omission flagged in earlier drafts and quantified in §8.10, but several spec features remain minimum-viable. The per-Jetty state machine carries a state byte but no transition logic for recoverable-fault send-queue drain; the exception-mode policy is not yet wired through; the asynchronous-event queue and the reserved public-Jetty identifiers are not implemented; and token-based access control at the receive boundary is currently enforced at memory-region granularity only, not per Jetty. Table 3 projects the per-Jetty byte cost of closing these gaps (20 B $\rightarrow$ 48 B; asymptotic ratio at $(1024, 1024)$ shifts from 4,855$\times$ to 3,855$\times$, so the state-scaling framing is robust). The target-side hardware dispatcher's RTL itself has not yet been independently placed-and-routed; it follows the same II=2 pattern as the four other gating elements (all of which close 322 MHz at II=2 per §4) and is in-line with §10's 13 KLUT ordering category as a follow-on item.

Comparison framing. OpenRoCE is an apples-to-apples baseline, not an optimised production stack. It anchors the comparison on identical infrastructure (same toolchain, same target, same harness). We do not address "how does OpenURMA compare to ConnectX-7?" — that would require disentangling the protocol design from a vendor's many-product-cycle optimisation budget. We address "how do UB's three commitments trade against the standard RoCE design point in synthesisable RTL on identical infrastructure?"

Why no SRNIC / StaR / 1RMA / Falcon / UEC in-fabric baselines. SRNIC [42], StaR [41], Aquila [7], and Falcon [38] are vendor-internal silicon; 1RMA [37] is a software stack atop commodity RoCE; UEC [40] is a specification without public RTL; and the IRN/MELO/-FaSR/DCP [32, 30, 10, 28] SR families are algorithm-and-simulation or vendor-internal hardware studies. None reduces to synthesisable FPGA RTL without effectively re-implementing it. We treat these as architectural reference points (§12, Table 4) rather than numerical baselines, and use OpenRoCE as the single in-fabric baseline because it is the only design point where every variable below the protocol can be held fixed.

12. Related Work

§2.2 grouped prior work on RDMA scale into three buckets — software above the device, hardware inside the device, and programmable substrates — and observed that none of the three removes both costs of the peripheral-NIC abstraction. We use the same lens to organise this section, then separately survey clean-slate competitor transports that, like UB, propose new abstractions rather than patch the old one. Table 4 summarises the landscape.

Software above the device. The earliest sustained attempts at RDMA scale worked entirely above the verb interface. FaSST [15] dropped to unreliable datagrams and reimplemented reliability in software, shrinking per-QP state at the cost of moving the reliability contract into the application. 1RMA [37] removed per-connection state entirely by issuing one-sided RDMAs over UDP with per-operation authentication, but the resulting protocol surface is read/write only and loses RC's per-connection ordering. Pony Express and the broader SmartNIC-pacing line take the same stance: the device is given, the work-queue / completion protocol is given, only what runs above can change. These approaches reduce one symptom (per-QP state) but leave the peripheral attachment and its four PCIe traversals in place.

Hardware inside the device. A long line of work has changed the NIC's internals while preserving the Queue-Pair-over-PCIe envelope. The connection- state direction — SRNIC [42] (on-chip cache with host-DRAM spill), StaR [41] (per-connection state reconstructed on demand from packet metadata) — reduces the on-chip footprint without changing what the QP fundamentally is. Meta's production RoCE deployment [6] reports needing up to 32 QPs per source–destination pair to approach roofline throughput, an empirical face of the same QP-binds-paths problem. The loss-recovery direction — IRN [32], MELO [30], FaSR [10], DCP [28], LEFT [11] — replaces Go-Back-N with bitmap-tracked selective retransmission and bounds the bitmap state. The multi-path direction — MP-RDMA [31], ConWeave [39], STrack [21], Spectrum-X [34] — admits packet spreading with bitmap-tracked per-transaction completion. These are real improvements within the abstraction, and UB inherits algorithmic ideas from them: its selective-acknowledgement path is the spec-level descendant of IRN, and the multi-path spreading reuses MP-RDMA's relaxed- ordering observation. The difference is structural: in UB the multi-path observa-

Table 4: Design-point positioning vs. recent RDMA scalability work (2018–2025). Rows are grouped by primary contribution. "Conn. state" is what the NIC stores per peer pair / per connection; "Loss recov." is the on-NIC loss-recovery scheme; "Multi-path" is whether the transport admits multi-path delivery without reorder breakage; "Ordering surface" is what semantic ordering the spec exposes; "Substrate" is the realisation. UB/OpenURMA is the only point that combines per-host-pair connection state, a four-axis opt-in ordering surface, and an open FPGA-RTL substrate.

Work	Conn. state	Loss recov.	Multi-path	Ordering surface	Substrate
RoCEv2 RC	per-QP	GBN	–	strict / QP	vendor ASIC
FaSST [15]	UD (per msg)	app-level	–	none	SW + commodity RNIC
1RMA [37]	none	per-op auth	yes	none	SW + RoCE
Aquila + 1RMA [7]	cell, none	cell-level	cell-network	none	custom ASIC
SRNIC [42]	QP cache + spill	RoCE	–	strict / QP	vendor ASIC
StaR [41]	reconstructed	RoCE	–	strict / QP	vendor ASIC
Falcon [38]	per-conn	SACK + RTT	yes	per-flow	vendor ASIC
UEC v1.0 [40]	packet-sprayed	SACK	yes	per-transaction	industry spec
Tonic [1]	programmable	programmable	programmable	programmable	FPGA (Verilog)
NanoTransport [14]	programmable	programmable	programmable	programmable	FPGA (Chisel/P4)
StRoM [36]	per-QP	RoCE-derived	–	per-QP	FPGA (HLS)
Flor [27]	per-QP	RoCE	–	per-QP	open framework
SwCC [9]	per-QP	RoCE + SW CC	–	per-QP	NIC + RISC-V
IRN [32]	per-QP	SR + per-pkt ACK	–	strict / QP	RoCE delta
MELO [30]	per-QP	const-mem SR	–	strict / QP	algorithm + sim
FaSR [10]	per-QP	line-rate SR	–	strict / QP	RNIC
DCP (SIGCOMM'25) [28]	per-QP	RTO-free SR	yes (pkt-LB)	strict / QP	hardware
LEFT [11]	per-QP	shared bitmap	yes	strict / QP	RNIC + sim
MP-RDMA [31]	per-QP + OOO	RoCE	yes	OOO + bitmap	RoCE delta
ConWeave [39]	per-QP	RoCE	yes	per-QP	switch + NIC
STrack [21]	per-flow	SR + fast recov	yes	per-flow	hardware
Spectrum-X [34]	per-QP	RoCE	yes	per-QP	NIC + switch
Justitia [43]	per-QP	–	–	–	SW userspace
Husky [19]	per-QP (diag)	–	–	–	test suite
Harmonic [29]	per-QP	–	–	–	hardware (BF-3)
SCR [44]	SW-programmable	SW-prog	SW-prog	SW-prog	BlueField-3 + DPA
UB / OpenURMA	**per host-pair**	**GBN + TPSACK**	**TPG + spray**	**four-axis opt-in**	**open FPGA RTL**

tion becomes an architectural default of the unordered service mode and is correctness-safe through the two-reorder-buffer split (§3.2) rather than through per-transaction bitmap state on the wire.

Programmable substrates. A third line builds FPGA or programmable-hardware fabrics that *host* arbitrary transports rather than proposing one. Tonic [1] (Verilog), NanoTransport [14] (P4/Chisel), StRoM [36] (HLS), Flor [27] (open heterogeneous-RNIC framework), SwCC [9] (on-NIC RISC-V for software-programmable congestion control), NICA [4], AccelNet [5], and ClickNP [24] (with KV-Direct [23] as an early ClickNP-style RDMA offload) are substrates on which new transports can be expressed. SCR [44] pushes the substrate idea into commercial silicon by exposing packet-granular software control on top of a vendor hardware transport. OpenURMA builds on OpenClickNP [22] as its substrate, but the contribution is not a new substrate; it is a new protocol expressed in synthesisable RTL on it, with per-element area, BRAM, and post-route timing numbers the architecture's costs can be argued from.

Clean-slate transport competitors. Three other proposals leave the QP-over-PCIe abstraction behind, with sharply different choices on what they replace it with. Aquila [7] pairs a custom intra-rack fabric with 1RMA cells: it removes per-host-pair state but its scope is a single clique and its design is fabric-specific. *Aquila bus-ified one rack; UB bus-ifies an arbitrary host that has the controller on-bus.* Google's Falcon [38] adds hardware-SACK loss recovery, RTT-based shaping, and PSP-encrypted multi-path but retains a per-connection abstraction in which connection state still fuses identity with transport. *Falcon optimises inside the per-pair envelope; UB removes the envelope.* Ultra Ethernet 1.0 [40] defines packet-sprayed, SACK-based reliable transport with per-transaction ordering guarantees implemented via per-transaction bitmap tracking on the wire. *UEC pays for multi-path safety with per-transaction SACK state; UB gets the same safety for free by separating its two reorder buffers.* The only shipping silicon for UB itself is Ascend 950 [13]; the silicon is closed and publishes no per-verb latency or per-element area, so the architectural choices are derivable from the spec but the implementation cost is not. OpenURMA fills that gap.

Memory-semantic fabrics. CXL [3] is a load/store fabric over PCIe physical layers; NVLink [33] is a vendor-proprietary intra-server fabric. The standard framing treats CXL as a niche distinct from scale-out RDMA, but a sharper framing is that the two are different layers of the same architectural axis: CXL provides memory-semantic access at intra-chassis distances; UB's load/store path provides memory-semantic access at cross-chassis distances. They compose. A future system can route the same ISA load instruction to CXL inside the chassis and to UB across the rack without an API change. The thing that permits the composition is UB's removal of the PCIe boundary on remote access; CXL by itself terminates at the chassis.

Multi-tenant isolation, congestion control, total order. Three further dimensions are tangential to the peripheral-NIC argument but worth noting briefly. Justitia [43], Husky [19], and Harmonic [29] attack RNIC tenant isolation in software and hardware respectively; the layer split removes one of the cache-pressure sources Husky exploits (the connection cache no longer scales with application-pair count). DCQCN [45] is RoCE's de-facto congestion control and serves as the OpenRoCE baseline's CC; UB defines its own queue-occupancy-based controller whose dynamics we measure in §9.7. At the opposite extreme from UB's opt-in ordering, 1Pipe [26] provides causally and totally ordered datacenter-wide communication at the cost of in-network barrier aggregation and an extra RTT per message; that contract is right for a higher-level transaction layer that may sit above URMA but wrong for the verb path itself, whose workloads are bandwidth-bound and per-pair-scoped.

13. Conclusion

OpenURMA is the first clean-room open implementation of the Unified Bus protocol's transport and transaction layers. The artifact realises the three architectural commitments the spec makes — a layer split that bounds per-NIC state additively in N and M, an ordering surface that applications opt into rather than always pay, and a load/store data path on the on-chip bus — in 39 synthesisable elements that close 322 MHz post-route on commodity FPGA, with a matched OpenRoCE baseline on the same toolchain, the same target part, and the same test harness.

What the open implementation tells us. The bounded state is asymptotic, not constant: $5\times$ less state than RoCEv2 RC at one local application and one remote endpoint, $4{,}855\times$ less at 1024 of each, crossing the on-chip-BRAM-to-host-DRAM spill boundary between $N{\approx}23$ and $N{\approx}1024$. The opt-in ordering surface costs 13 KLUT (10.6% of the design's LUT budget) and *zero* pipeline cycles on the operations that do not request ordering; the cost is paid only on opt-in. The load/store path delivers its first wire flit in 8 cycles ($\approx$25 ns) at 322 MHz, and the full CPU-to-remote-DRAM-to-CPU round trip in the two-node simulator measures $\approx$500 ns on a 64-byte READ — $4.37\times$ below the matched RoCE-DMA baseline (2186 ns). The gap is structural: the nine PCIe-bound cost components on the RoCE round trip are protocol consequences of disjoint CPU/NIC address spaces, and the load/store path removes the disjunction. The whole design fits in roughly 14% of an Alveo U50's LUT budget.

Why the open artifact matters. The architecture is Huawei's. Ascend 950 ships it in commodity silicon. What did not exist until now is a way for the community to *measure* the architecture — compare its operating points against RoCE on identical infrastructure, locate the workloads where it wins or loses, and prototype follow-on transports against UB-style assumptions. OpenURMA closes that gap. The implementation makes visible a coupling the spec asserts but cannot prove on paper: each of the three architectural commitments depends on the others being present, and removing any one while keeping the others on a PCIe-attached NIC reproduces RoCE's costs.

What we hand to follow-on work. The artifact is released as a substrate for further investigation: multi-flit retransmission and an alternative congestion-control algorithm are flagged in §11 as the most obvious near-term extensions; the gem5 full-system scaffold is intended to support comparison of UB against Falcon, UEC, and SRNIC/StaR-derivative transports on the same infrastructure; and the matched OpenRoCE baseline is intended to provide a controlled reference any of those comparisons can re-use. The questions Unified Bus raises — about state scaling, about ordering surfaces, about memory-semantic data paths in datacenter fabrics — can now be investigated outside the closed silicon that currently ships.

Code. `https://github.com/bojieli/OpenURMA`.

Acknowledgments. This work builds on the OpenClickNP toolchain (an open re-implementation of the ClickNP element model) and reads the published UB-Base-Specification 2.0.1 as authoritative for the protocol; the implementation choices, modeling assumptions, and measurement framework are ours, and any deviations from the spec are unintentional.

References

[1] Mina Tahmasbi Arashloo, Alexey Lavrov, Manya Ghobadi, Jennifer Rexford, David Walker, and David Wentzlaff. Enabling programmable transport protocols in high-speed NICs. In *Proc. USENIX NSDI*, 2020.

[2] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with YCSB. In *Proc. ACM SoCC*, 2010. The Yahoo! Cloud Serving Benchmark; we use the YCSB-A 50/50 Get-Put Zipfian workload in §9.8.

[3] CXL Consortium. Compute Express Link (CXL) Specification 3.1. `https://www.computeexpresslink.org/`, 2024.

[4] Haggai Eran, Lior Zeno, Maroun Tork, Gabi Malka, and Mark Silberstein. NICA: An infrastructure for inline acceleration of network applications. In *Proc. USENIX ATC*, 2019.

[5] Daniel Firestone, Andrew Putnam, Sambhrama Mundkur, Derek Chiou, Alireza Dabagh, Mike Andrewartha, Hari Angepat, et al. Azure Accelerated

Networking: SmartNICs in the public cloud. In *Proc. USENIX NSDI*, 2018.

[6] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. RDMA over Ethernet for distributed training at meta scale. In *Proc. ACM SIGCOMM*, 2024.

[7] Dan Gibson, Hema Hariharan, Eric Lance, Moray McLaren, Behnam Montazeri, Arjun Singh, Stephen Wang, Hassan M. G. Wassel, Zhehua Wu, Sunghwan Yoo, Raghuraman Balasubramanian, Prashant Chandra, Michael Cutforth, Peter Cuy, David Decotigny, Rakesh Gautam, Alex Iriza, Milo M. K. Martin, Rick Roy, Zuowei Shen, Ming Tan, Ye Tang, Monica Wong-Chan, Joe Zbiciak, and Amin Vahdat. Aquila: A unified, low-latency fabric for datacenter networks. In *Proc. USENIX NSDI*, 2022.

[8] Tingbo He. A time scaling theory for multilayer electronic systems. *ChinaXiv*, May 2026. chinarxiv-202605.00224. Perspective from Huawei Semiconductor: τ scaling as successor to geometric Moore's-Law scaling; positions Unified Bus as the system-layer τ reduction mechanism with end-to-end remote-access latency from $\sim$10s of μs (TCP/IP-class) to $\sim$100 ns.

[9] Hongjing Huang, Jie Zhang, Xuzheng Chen, Ziyu Song, Jiajun Qin, and Zeke Wang. SwCC: Software-programmable and per-packet congestion control in RDMA engine. In *Proc. USENIX ATC*, 2025.

[10] Peihao Huang, Guo Chen, Xin Zhang, Can Liu, Hongyu Wang, Huijun Shen, Ying Bian, Yuanwei Lu, Zhenyuan Ruan, Bojie Li, Jiansong Zhang, Yongfeng Liu, and Zhigang Chen. Fast and scalable selective retransmission for RDMA. In *Proc. IEEE INFOCOM*, 2025.

[11] Peihao Huang, Xin Zhang, Zhigang Chen, Can Liu, and Guo Chen. LEFT: Lightweight and fast packet reordering for RDMA. In *Proc. APNet*, 2024.

[12] Huawei Technologies. UB-base-specification 2.0.1. `https://www.unifiedbus.org/`, 2024. Unified Bus consortium specification, available from the consortium's documentation portal.

[13] Huawei Technologies. Ascend 950 NPU architecture white paper. Huawei vendor white paper, May 2026. Architectural disclosure for the Ascend 950PR and 950DT NPUs; first publicly documented silicon implementing the Unified Bus spec, with URMA (asynchronous Write/Read/Send/Atomic via Jetty) and UB Memory (synchronous Load/Store + AtomicStore/Load/Swap/CAS) exposed as distinct on-die paths, plus URMA-CTP / URMA-TP transport modes, UBoE 2×400 Gbps, a hardware Collective Communication Unit (CCU), UB On-Chip Switch for in-die forwarding, and a UB super-node target of 8192 cards (cluster target >128K).

[14] Stephen Ibanez, Alex Mallery, Serhat Arslan, Theo Jepsen, Muhammad Shahbaz, Nick McKeown, and Changhoon Kim. NanoTransport: A low-latency, programmable transport layer for NICs. In *Proc. ACM SOSR*, 2021.

[15] Anuj Kalia, Michael Kaminsky, and David G. Andersen. Using RDMA efficiently for key-value services. In *Proc. ACM SIGCOMM*, 2014.

[16] Anuj Kalia, Michael Kaminsky, and David G. Andersen. FaSST: Fast, scalable and simple distributed transactions with two-sided (RDMA) datagram RPCs. In *Proc. USENIX OSDI*, 2016.

[17] Anuj Kalia, Michael Kaminsky, and David G. Andersen. Datacenter RPCs can be general and fast. In *Proc. USENIX NSDI*, 2019.

[18] Antoine Kaufmann, Tim Stamler, Simon Peter, Naveen Kr. Sharma, Arvind Krishnamurthy, and Thomas Anderson. TAS: TCP acceleration as an OS service. In *Proc. EuroSys*, 2019. Reports detailed PCIe-class transaction latency decompositions used as parameter references in §8.

[19] Xinhao Kong, Jingrong Chen, Wei Bai, Yechen Xu, Mahmoud Elhaddad, Shachar Raindel, Jitendra Padhye, Alvin R. Lebeck, and Danyang Zhuo. Understanding RDMA microarchitecture resources for performance isolation. In *Proc. USENIX NSDI*, 2023.

[20] Xinhao Kong, Yibo Zhu, Huaping Zhou, Zhuo Jiang, Jianxi Ye, Chuanxiong Guo, and Danyang Zhuo. Collie: Finding performance anomalies in RDMA subsystems. In *Proc. USENIX NSDI*, 2022.

[21] Yanfang Le, Rong Pan, Peter Newman, Jeremias Blendin, Abdul Kabbani, Vipin Jain, Raghava Sivaramu, and Francis Matus. STrack: A reliable multipath transport for AI/ML clusters. arXiv:2407.15266, 2024.

[22] Bojie Li. OpenClickNP: a clean-room reimplementation of ClickNP on Alveo U50. `https://github.com/bojieli/OpenClickNP`, 2025–2026.

[23] Bojie Li, Zhenyuan Ruan, Wencong Xiao, Yuanwei Lu, Yongqiang Xiong, Andrew Putnam, Enhong Chen, and Lintao Zhang. KV-Direct: High-performance in-memory key-value store with programmable NIC. In *Proc. ACM SOSP*, 2017.

[24] Bojie Li, Kun Tan, Layong Larry Luo, Yanqing Peng, Renqian Luo, Ningyi Xu, Yongqiang Xiong, Peng Cheng, and Enhong Chen. ClickNP: Highly flexible and high-performance network processing with reconfigurable hardware. In *Proc. ACM SIGCOMM*, 2016.

[25] Bojie Li, Zhilong Xiang, Xiang Wang, Hongru Jonathan Zhou, and Kun Tan. FastWake: Revisiting host network stack for interrupt-mode RDMA. In *Proc. APNet*, 2023.

[26] Bojie Li, Gefei Zuo, Wei Bai, and Lintao Zhang. 1Pipe: Scalable total order communication in data center networks. In *Proc. ACM SIGCOMM*, 2021.

[27] Qiang Li, Yixiao Gao, Xiaoliang Wang, Haonan Qiu, Yanfang Le, Derui Liu, Qiao Xiang, Fei Feng, Peng Zhang, Bo Li, Jianbo Dong, Lingbo Tang, Hongqiang Harry Liu, Shaozong Liu, Weijie Li, Rui Miao, Yaohui Wu, Zhiwu Wu, Chao Han, Lei Yan, Zheng Cao, Zhongjie Wu, Chen Tian, Guihai Chen, Dennis Cai, Jinbo Wu, Jiaji Zhu, Jiesheng Wu, and Jiwu Shu. Flor: An open high performance RDMA framework over heterogeneous RNICs. In *Proc. USENIX OSDI*, 2023.

[28] Wenxue Li, Xiangzhou Liu, Yunxuan Zhang, Zihao Wang, Wei Gu, Tao Qian, Gaoxiong Zeng, Shoushou Ren, Xinyang Huang, Zhenghang Ren, Bowen Liu, Junxue Zhang, Kai Chen, and Bingyang Liu. Revisiting RDMA reliability for lossy fabrics. In *Proc. ACM SIGCOMM*, 2025. Best Student Paper, Honorable Mention.

[29] Jiaqi Lou, Xinhao Kong, Jinghan Huang, Wei Bai, Nam Sung Kim, and Danyang Zhuo. Harmonic: Hardware-assisted RDMA performance isolation for public clouds. In *Proc. USENIX NSDI*, 2024.

[30] Yuanwei Lu, Guo Chen, Bojie Li, Kun Tan, Yongqiang Xiong, Peng Cheng, Jiansong Zhang, Enhong Chen, and Thomas Moscibroda. Memory efficient loss recovery for hardware-based transport in datacenter. In *Proc. APNet*, 2017.

[31] Yuanwei Lu, Guo Chen, Bojie Li, Kun Tan, Yongqiang Xiong, Peng Cheng, Jiansong Zhang, Enhong Chen, and Thomas Moscibroda. Multi-Path transport for RDMA in datacenters. In *Proc. USENIX NSDI*, 2018.

[32] Radhika Mittal, Alexander Shpiner, Aurojit Panda, Eitan Zahavi, Arvind Krishnamurthy, Sylvia Ratnasamy, and Scott Shenker. Revisiting network support for RDMA. In *Proc. ACM SIGCOMM*, 2018.

[33] NVIDIA Corporation. NVLink: A high-bandwidth inter-GPU interconnect. Vendor whitepaper, 2014–2025. Successive generations of the NVLink fabric are described in the NVIDIA whitepaper series.

[34] NVIDIA Networking. NVIDIA Spectrum-X: Adaptive routing and telemetry-based congestion control for AI networks. NVIDIA technical brief, 2024. Vendor description of multi-path adaptive-routing delivery over Spectrum-4 / BlueField-3 NICs; the closest commercially-deployed point of comparison to UB's TPG multi-path scheme.

[35] Sebastian Ramos and Torsten Hoefler. Designing high-performance, low-latency multi-cluster communication on modern InfiniBand networks. In *Proc. ACM HPDC*, 2023. Reports ConnectX-7 PCIe round-trip latencies in the ∼300–500 ns range; we use this as the parameterised PCIe RTT in §8.

[36] David Sidler, Zeke Wang, Monica Chiosa, Amit Kulkarni, and Gustavo Alonso. StRoM: Smart remote memory. In *Proc. EuroSys*, 2020.

[37] Arjun Singhvi, Aditya Akella, Dan Gibson, Thomas F. Wenisch, Monica Wong-Chan, Sean Clark, Milo M. K. Martin, Moray McLaren, Prashant Chandra, Rob Cauble, et al. 1RMA: Re-envisioning remote memory access for multi-tenant datacenters. In *Proc. ACM SIGCOMM*, 2020.

[38] Arjun Singhvi, Nandita Dukkipati, Prashant Chandra, Hassan M. G. Wassel, Naveen Kr. Sharma, Anthony Rebello, Henry Schuh, Praveen Kumar, Behnam Montazeri, Neelesh Bansod, Sarin Thomas, Inho Cho, Hyojeong Lee Seibert, Baijun Wu, Rui Yang, Yuliang Li, Kai Huang, Qianwen Yin, Abhishek Agarwal, Srinivas Vaduvatha, Weihuang Wang, Masoud Moshref, Tao Ji, David Wetherall, and Amin Vahdat. Falcon: A reliable, low latency hardware transport. In *Proc. ACM SIGCOMM*, 2025. Specification contributed to OCP 2024.

[39] Cha Hwan Song, Xin Zhe Khooi, Raj Joshi, Inho Choi, Jialin Li, and Mun Choon Chan. Network load balancing with in-network reordering support for RDMA. In *Proc. ACM SIGCOMM*, 2023.

[40] Ultra Ethernet Consortium. Ultra Ethernet specification 1.0. Industry specification, 2025. Released June 2025 under Linux Foundation JDF; `https://ultraethernet.org/`.

[41] Xizheng Wang, Guo Chen, Xijin Yin, Huichen Dai, Bojie Li, Binzhang Fu, and Kun Tan. StaR: Breaking the scalability limit for RDMA. In *Proc. IEEE ICNP*, 2021.

[42] Zilong Wang, Layong Luo, Qingsong Ning, Chaoliang Zeng, Wenxue Li, Xinchen Wan, Peng Xie, Tao Feng, Ke Cheng, Xiongfei Geng, Tianhao Wang, Weicheng Ling, Kejia Huo, Pingbo An, Kui Ji, Shideng Zhang, Bin Xu, Ruiqing Feng, Tao Ding, Kai Chen, and Chuanxiong Guo. SRNIC: A scalable architecture for RDMA NICs. In *Proc. USENIX NSDI*, 2023.

[43] Yiwen Zhang, Yue Tan, Brent Stephens, and Mosharaf Chowdhury. Justitia: Software multi-tenancy in hardware kernel-bypass networks. In *Proc. USENIX NSDI*, 2022.

[44] Chenxingyu Zhao, Jaehong Min, Ming Liu, and Arvind Krishnamurthy. White-boxing RDMA with packet-granular software control. In *Proc. USENIX NSDI*, 2025.

[45] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. Congestion control for Large-Scale RDMA deployments. In *Proc. ACM SIGCOMM*, 2015.